

関西文芸交流会 情報リテラシ研修

実務者・技術者向け資料

関西文芸交流会 技術開発部 (KLEA ITDD)

初版 (v1.0)
2026 年 7 月 20 日

はじめに

本カリキュラムは、関西文芸交流会の実務者および技術者向けに、業務上必要な情報リテラシ・IT・セキュリティ関連の知識を体系的に学び、現実に応時適切な対応をとることができるよう研修することを目的として開発されました。

個人個人がメディアとなり得る現代において、また著作物や個人情報を扱う団体にあって、正しい知識を身につけ理解することは重要であり、またそれ以上に、それらを実際に活かすことができることが不可欠となりつつあります。そのため、本資料では、可能な限り実例を盛り込み、それぞれの項目が単に紙面上のものではなく、現実に応大きな問題を引き起こす可能性をはらんでいること、またそれらにどう対処すべきかを具体的にイメージできるように配慮しました。

情報リテラシやセキュリティに関するスタンダードは日進月歩であり、適時に情報をアップデートすることが必要です。

第1章・第2章では、原理原則や基本となる考え方について整理しています。第3章から第6章では、個人情報、著作権、メール、SNS・公開情報の扱いなど、実際の活動に大きくかわかる項目を確認します。また、第7章では生成AIに関する指針を扱います。第8章・第9章では主に技術者向けに、ファイル管理、バックアップ、ログ、Git、サーバ・コーディング関係の実務上の注意点を整理します。

なお、本資料は高校「情報I」の内容を前提としていますので、知識に不足がある場合は参照するとよいでしょう。

目次

第1章	フェイルセーフとインシデント対応	1
第2章	情報リテラシ概論	7
第3章	個人情報	18
第4章	著作権	22
第5章	メール	25
第6章	SNS・公開情報・テストデータ管理	29
第7章	生成AIと情報管理	34
第8章	ファイル管理・バックアップ・ログ・Git	37
第9章	サーバ・コーディング関係のセキュリティ	43
	確認の解答	50

第 1 章 フェイルセーフとインシデント対応

情報セキュリティに関する有名なジョークとして、「最大の脆弱性はキーボードとイスの間にある」というものがあります。人間こそが最大の脆弱性だということです。しかし、これは決して笑い話ではありません。半数以上の情報漏洩やセキュリティインシデントは実際にヒューマンエラーによって引き起こされているという統計すらあるからです。

そのため、人間という、最大かつ完全にはなくすことのできない脆弱性になるだけ対処すべく、情報セキュリティの世界で広く共有されているコンセプトがあります。それはヒューマンエラーに対する考え方とフェイルセーフの概念です。本章では、各トピックに通底する、非常に基礎的かつ重要なこのアイデアを扱います。

1.1 ヒューマンエラーはなくなるしない

どれだけ注意深い人であっても、人間は必ずミスを行います。しかも、そのミスの中には「なぜそんなことが起きたのか理解できない」「常識的に考えてあり得ない」と思われるものが、一定の確率で含まれます。これは能力や性格の問題ではなく、人間の認知や判断の仕組みに由来するものです。

ヒューマンエラーの例：

- 共通テスト本番で、数学 I + A と間違えて数学 I を解答してしまう
- メールの宛先を確認したつもりで、誤った相手に送信する
- 重要なファイルを誤って削除する
- 公開してはいけない設定を誤って有効にしてしまう

これらは「あり得ないミス」ではなく、「人間であれば起こり得るミス」です。情報管理やセキュリティ対策を考えるうえで最初に必要なのは、人間は必ずミスをする存在であることを前提にすることです。

1.1.1 現実の事例：羽田空港航空機衝突事故（2024 年）

2024 年 1 月 2 日、羽田空港の滑走路上で海上保安庁機と日本航空機が衝突し、海上保安庁機の乗員 5 人が死亡しました。日本航空機では乗客・乗員 379 人全員が脱出しました。

事故後に国土交通省がまとめた再発防止策では、滑走路への誤進入を「防ぐ」、誤進入に「気づく」、航空機へ「知らせる」という多段階の対策、注意喚起システムや管制体制の強化などが示されました。

単一の聞き間違いや確認漏れがそのまま衝突まで進まない設計になっていたが、というフェイルセーフと多重防護の代表的な資料です。事故調査では、個人の責任だけに原因を求めず、管制、表示、通信、機器、判断など複数の要素を検証する必要があります。

1.2 ミスに対する正しい対策

ミスに対してよく取られる対策の多くは、上記の前提を無視しています。

誤った対策の例：

- 「気をつける」
- 「注意を徹底する」
- 「ミスをした人にペナルティを設ける」
- 「とりあえず多人数で確認する」

なぜなら、どれだけ気をつけても人間はミスをするからです。不要なペナルティは報告を遅らせ、隠蔽を生み、結果として被害を拡大させます。

正しい対策：

- そもそもミスが起こりにくい環境を作る
- ミスをしても被害が小さくて済む仕組みを作る

これが フェイルセーフ の考え方です。個人の注意力ではなく、構造や仕組みでミスを吸収することが重要です。

1.2.1 現実の事例：JR 西日本福知山線列車脱線事故と日勤教育（2005 年）

JR 西日本では、事故や規程違反を起こした乗務員に対する再教育として、過酷な「日勤教育」が行われていました。事故調査報告書では、当時の再教育について、精神論的な教育に偏らず、事故防止にふさわしい内容へ見直す必要があるとされています。

この事例は、「ミスをした人を強く責めれば安全になる」という考え方の危うさを示します。過度に懲罰的な環境では、ミスや遅れを早く報告するよりも、隠したり一人で取り返そうとしたりする行動が起きやすくなります。その結果、周囲が支援・判断できる機会を失い、別の危険を増やしかねません。

日勤教育だけを事故の原因とみなすことはできません。しかし、安全対策は個人への反省を求めるだけで終わらせず、ミスを報告して支援を受けられる環境、無理な挽回を必要としない運用、誤りが重大事故へ進まない設備・手順を組み合わせる必要があります。

1.3 冗長性

フェイルセーフを実現する基本的な考え方のひとつが、冗長性です。冗長性とは、必要な機能や情報をひとつだけに依存させず、予備・代替・複数経路を持たせることです。

たとえば、次のような状態は冗長性がありません。

- 重要なファイルが、担当者 1 人の PC にしかない
- 連絡先や手順書が、特定のチャット履歴の中にしかない
- 本番環境に詳しい人が 1 人しかない

このような状態では、その 1 人が不在になったり、PC が壊れたり、アカウントに入れなくなったりただけで業務が止まります。これを単一障害点と呼びます。単一障害点を減らすことは、情報管理における基本的な安全設計です。

冗長性の例：

- 重要ファイルは共有ストレージに置き、権限を管理する
- バックアップを複数世代で保存する
- 団体アカウントには複数の管理者を置く
- 2FA のバックアップコードや復旧手順を安全な場所に保管する
- 連絡先、手順、権限移譲の方法を文書化する
- 重要な運用は、少なくとも 2 人以上が実行・確認できる状態にする

ただし、冗長性の確保はセキュリティリスクの増加につながることもあります。例えば、機密情報や個人情報を不用意に複製すると、漏洩経路が増えます。冗長性を持たせるときは、必要な範囲で、管理された形で、復旧できる状態にすることが重要です。

実務では、次の順で考えます。

1. 失われると止まるものは何か

2. それは 1 人・1 台・1 か所・1 つのサービスに依存していないか
3. 代替手段や復旧手順はあるか
4. その代替手段は、定期的に確認されているか

冗長性があるかどうかは、平常時には見えにくいものです。しかし、端末故障、アカウントロック、サービス停止、災害、誤削除などが起きたときに、その差が明確に現れます。

1.4 ミス対策の実例

たとえば鉄道では、運転士が停止信号を見落としたり誤認したりする可能性を前提に、自動列車停止装置 (ATS) が使われています。停止信号を越えて進もうとした場合や、制限速度を超えようとした場合に、自動的にブレーキを作動させます。

実際の仕組み：

- 運転士が停止信号を誤認しても、装置がブレーキを作動させて停止信号の手前で止める
- 曲線や分岐器などの速度制限を超えそうな場合にも、装置が減速・停止を支援する
- 運転士の注意力だけではなく、信号・速度・ブレーキを連動させて危険な状態への移行を防ぐ

これは、運転士が信号を誤認しないよう教育するだけではなく、誤認が起きても列車を危険な状態へ進ませない仕組みです。完全に事故をなくすものではありませんが、人のミスの結果を安全側に閉じ込めるという点で、フェイルセーフの考え方を具体化しています。この考え方は、分野が違って共通するものです。

1.5 ミス対策とトレード・オフ

ミス対策は無制限に行えばよいものではありません。対策にはコストや手間がかかり、それ自体が別の問題を生むことがあります。

不適切なミス対策の例：

- 数学の試験で、計算ミスを防ぐためにすべての計算について 2 通り以上の方法で検算を行った結果、時間が足りずに 30 点しか取れなかった。
- お金を正確に管理するために、1 年あたり 1 億円するサービスを導入した結果、コストのかかり過ぎによって経営が傾いた。
- 判断ミスを防ぐために稟議制を導入した結果、意思決定のプロセスに時間がかかり、時代に残された。

そのため、ミス対策では次の判断が重要です。

- コストがほとんどかからない対策は必ず行う
- 想定される被害の大きさと、対策コストが釣り合うかを考える

適切なミス対策の例：

- 計算ミスを防ぐために、紙のあちこちに計算過程を書くのではなく、同じ問題の計算は一か所にまとめて書くことにした。(コスト 0 の対策)
- SNS での炎上を避けるため、事前に登録した禁止語が含まれる内容の投稿は自動的に中止されるようなシステムを構築した。(初期投資が必要だが、運用コストが 0 の対策)
- 機密情報の漏洩を避けるため、重要なファイルには会社には設置されているマシンからしかアクセスできないようにした。(不便さというコストはあるが、甚大な被害を防ぐ対策)

1.5.1 現実の事例：パロマ湯沸器一酸化炭素中毒事故

対策コストに関する示唆的な事故です。パロマ工業製の半密閉式ガス瞬間湯沸器では、安全装置が作動して機器が停止するのを避けるため、修理業者らが安全装置を迂回する不正改造を行い、一酸化炭素中毒による死亡事故が続きました。

「安全装置が邪魔だから無効化する」という運用が、フェイルセーフを機能不全に陥らせた典型例です。安全装置を設けるだけでなく、容易に無効化できない設計、修理作業の管理、同種事故を組織全体で集約して利用停止や点検につなげる仕組みまで必要です。また、こうした迂回を防ぐため、安全装置そのものについても、安全性を担保しながら、なるべく不便さを増大させない設計にすることも重要といえるでしょう。

1.6 簡易リスク評価

トレード・オフを判断するためには、「何がどれくらい危ないか」を見積もる視点が必要です。簡易な方法として、次の考え方が使われます。

- リスク = 起きる確率 × 被害の重大度

ここでいう「確率」や「重大度」は、厳密な数値である必要はありません。

例1：

- ほかのサービスと連携していない、どうしてもよいゲームのアカウントで、よくある弱いパスワード（例：123456等）を使っている
- 確率：10 × 被害の重大度：1 = リスク：10（小）

例2：

- クレジットカードで同じ暗証番号を使いまわしている
- 確率：10 × 被害の重大度：1000 = リスク：10000（大）

例3：

- わずかな確率で死亡する治験アルバイト
- 確率：0.0001 × 被害の重大度：∞ = リスク：∞（甚大）

1.7 インシデント対応の基本

インシデントとは、情報事故・セキュリティ事故・その疑いを指します。（注：インシデントは分野により用法が揺れます。本資料では、実害の有無にかかわらず「対応が必要な事故・疑い」をインシデントと呼びます。単なる観測された出来事は（セキュリティ）イベントと呼ぶことがあります。また「アクシデント」は日常語としての事故を指すことはありますが、情報セキュリティの実務ではインシデントの語が一般的なため、本資料では原則用いません。）

インシデントの例：

- 個人情報情報を誤送信した
- 共有リンクを誤って公開した
- 団体アカウントの乗っ取りが疑われる
- 機密情報を誤って公開した
- 不審な添付ファイルを開いてしまった

重要なのは、原因究明よりも、まず被害拡大を止め、関係者が同じ状況認識を持つことです。

1.8 初期対応の優先順位

インシデント対応は、原則として次の順序で行います。

1. 止血：被害拡大を止める（技術的に止める）
2. 共有：関係者に速やかに共有する（隠さない）
3. 記録：何が起きたかを時系列で残す（後で必ず必要になる）
4. 切り分け：影響範囲を把握する（誰に・何が・どこまで）
5. 再発防止：仕組みを変える（注意喚起だけで終わらせない）

一方、以下のような対応を行うと、被害が拡大しやすくなります。

誤った対応の例：

- 自分だけで抱え込む（報告を先延ばしにする）
- 原因が確定するまで共有しない
- 証跡（ログや通知、画面）を消す
- 憶測の対外発信をする

特に、「原因が分かってから報告する」「整理できてから共有する」という考え方は、一見正しそうに見えても、実際にはその間に被害が拡大しやすくなるため、適切とはいえません。

実際の報告先や連絡経路は、組織や業務ごとに異なりますが、迷ったときほど、定められた報告先や責任者に早く共有するという原則を理解することが重要です。

1.9 初動で考える順序

事故の種類によって細部は異なりますが、混乱した場面でも「被害を広げない」「一人で抱えない」「後から確かめられるようにする」ことを原則とすることが重要です。次の項目は、その判断を具体化した例です。

- 影響範囲が広がる操作を止める（投稿停止、共有停止、権限付与の停止など）
- 共有リンクの無効化、アクセス権の剥奪
- アカウント侵害が疑われる場合は、ログイン停止・パスワード変更・2FA 確認
- Secrets が漏れた場合は、当該キー/トークンを無効化・再発行（第 9 章 サーバ・コーディング関係のセキュリティ）
- いつ・何が・どこで起きたかをメモする（時刻、対象、操作、相手）
- 関係者へ速やかに共有する

1.10 報告に必要な項目

報告に含めるべき項目の例として、以下のようなものがあげられます。

- 発生日時・発覚日時
- 対象となる範囲
- 事実関係
- 現時点の影響

可能であれば、いわゆる 5W1H を含めれば十分といえるでしょう。ここで、これらの項目の一部が不明であっても、その項目の解明を行うよりは、不明のまま共有するほうが適切な対応といえます。確定させる前に、被害拡大を止めることが優先だからです。

1.11 参考資料

- 運輸安全委員会「航空事故調査について（経過報告）」*1
- 運輸安全委員会「福知山線列車脱線事故に係る鉄道事故調査報告書の概要」*2
- 国土交通省「用語解説ページ：自動列車停止装置 (ATS)」*3
- 経済産業省「パロマ工業株式会社製半密閉式ガス瞬間湯沸器による一酸化炭素中毒事故」*4

1.12 確認

1. フェイルセーフとはどのような考え方か。適切な実例を用いて説明せよ。
2. インシデントの初期対応で行うべきことは何か。また、原因が分かってから報告する対応が危険な理由は何か。
3. ミスをした人を強く責めるだけの対策が、かえって安全を損なうのはなぜか。適切な実例を用いて説明せよ。

*1 https://jtsb.mlit.go.jp/aircraft/rep-acci/keika20241225-JA722A_JA13XJ.pdf

*2 <https://jtsb.mlit.go.jp/fukuchiyama/kensyou/fu04-finalreport-annex-2-1.pdf>

*3 <https://www.mlit.go.jp/yougo/j-s1.html>

*4 <https://www.meti.go.jp/policy/consumer/seian/gasji/Paloma/index.html>

第 2 章 情報リテラシ概論

この章では、基本的な情報リテラシに関するトピックを扱います。多くは高校「情報 I」で扱われる内容ですが、こうした基礎的な内容でも日々アップデートされているものです。いま一度確認しておきましょう。

2.1 Web 上の住所：URL とドメイン

Web サイトやファイルの場所を示す文字列を URL といいます。URL は、おおむね「どの方式で通信するか」「どの相手に接続するか」「その中のどこを開くか」から成り立っています。

```
<https://www.example.com/event/guide.html>
|           |           |
|           |           |—— サイト内の場所（パス）
|           |—— 接続先の名前（ホスト名、中心となるドメインは example.com）
|—— 通信方式（スキーム）
```

ドメインは、example.com のような、人が読めるインターネット上の名前です。DNS という仕組みが、この名前を通信先の IP アドレスへ対応付けます。www.example.com と example.com は似ていますが、前者の www はサブドメインであり、運営者は別の接続先に設定できます。そのため、サイト名やロゴではなく、中心となるドメインを文字どおり確認することが重要です。

2.2 通信方式（スキーム）が示すもの

URL の先頭にある https や ftp は、通信の約束事（プロトコル）を示します。これは「どのように通信するか」を表すものであり、サイトや相手が信用できるかを直接示すものではありません。

- **https**：Web ページを、TLS による暗号化を用いて送受信する方式です。通信の盗聴や改ざんを防ぐうえで重要ですが、偽サイトでも HTTPS を使えるため、運営者が正しいことまでは保証しません。
- **http**：暗号化されない Web 通信です。ログイン情報や個人情報を入力するサイトで使うべきではありません。

なお、ブラウザのアドレス欄などに表示される鍵マーク (HTTPS) は、「通信が暗号化されている」ことを示すだけで、「そのサイトが安全」「運営者が正しい」ことを保証しません。偽サイトでも HTTPS は取得できます。そのため、「鍵マークがあるから安全」という判断は誤りです。

このほか、古いファイル送受信の方式である ftp や、それを発展させたファイル転送方式の sftp、ftps などがあります。

2.3 URL 偽装

Web サイトやメールの見た目は、いくらでも本物そっくりに作れます。そのため、「画面の雰囲気」「ロゴ」「文章の丁寧さ」だけで本物かどうかは判断できません（もちろん、見た目が怪しいサイトやメールが本物でないであろうことはいうまでもありません。逆にいえば、サイトやメールを作成する際に見た目に注意を払うことは重要であり、またはある種の責務ともいえるでしょう）。

特に注意すべきなのは URL（リンク先）です。

- <https://official-example.com>

- <https://official.example.com>
- <https://example-official.com> これらは一見似ていますが、いずれもまったく別のサイトです。重要なのは、見た目ではなく、**URL そのものを確認すること**です。特にスマートフォンなどの画面が小さい機器では、ブラウザのアドレス欄が非表示になっているケースも多く、一層の注意を要します。

2.4 検索結果の一番上でも安全とは限らない

検索結果は、サイトの安全性を保証する順番で並んでいるわけではありません。検索語との関連性、広告出稿、検索エンジンの評価などにより表示されるため、偽サイトや詐欺サイトが上位に現れることがあります。

特に注意する点：

- 最上位にあるという理由だけで公式サイトだと判断しない
- サイト名ではなく、ドメインを含む URL を確認する
- よく使う重要なサイトは、確認済みの公式ページをブックマークする
- 公式アプリがある場合は、公式ストアから導入したアプリを利用する

検索結果から開く場合も、ログイン情報やカード情報を入力する前に、公式サイトが案内している URL と一致するかを確認します。

2.4.1 現実の事例：検索結果最上部に表示された「えきねっと」偽サイト（2022 年）

2022 年、Google で JR 東日本の予約サービス「えきねっと」を検索すると、検索結果最上部の偽サイトが表示される事例が発生しました。偽サイトは公式サイトに似た画面や駅名検索機能を備え、ログイン画面へ誘導し、認証情報や決済情報の窃取を試みていました。

この事例から学ぶ点：

- 「検索して一番上を開く」は、公式サイトの確認方法として十分ではない
- 広告枠にも偽サイトが掲載される場合がある
- 本物そっくりに操作できても、正規サイトである証拠にはならない

2.5 サードパーティーによる短縮 URL の危険性

サードパーティーが提供する短縮 URL (bit.ly など) は、実際のリンク先が見えません。そのため、意図しないサイトへ誘導されやすく、また URL の偽装にも気づきにくくなります。そのほか、短縮 URL に関して特に問題になるのは、次の点です。

- 開く前に本来のドメインを確認しにくい
- 転送先が後から変更される、または複数段階のリダイレクトで追跡しにくくなる
- アクセス解析やトラッキングに使われ、誰がいつ開いたかが第三者サービスに残ることがある
- 短縮 URL サービス自体が停止したり、仕様変更・アカウント停止・有料化によりリンク切れることがある
- 組織の公式発信に使うと、受け手が「正規の URL かどうか」を判断しにくくなる

短縮 URL は、SNS など文字数制限がある場面では便利ですが、セキュリティ上は大きなリスクでしかありません。特に、ログイン、支払い、個人情報入力、ファイルダウンロードにつながるリンクでは避けるべきです。

リンクを受け取ったときの判断：

- 短縮 URL は安易に開かない
- 開く必要があるときは、送信者に別経路で確認する
- 可能なら短縮前の URL を提示してもらう
- 開いた後にログインや支払いを求められたら、そのページでは入力せず、公式サイトや公式アプリから開き直す

リンクを共有する側の責任：

- 公式案内、申込フォーム、決済、ログイン、重要資料の共有には短縮 URL を使わない
- どうしても使う場合は、短縮前の URL や公式ドメイン名も併記する
- 長期保存する文書、印刷物、規約、マニュアルでは短縮 URL を避ける
- 期限付き・一時的な案内に使う場合も、リンク切れ時の代替導線を用意する
- 外部短縮サービスではなく、自組織の管理下にあるリダイレクト URL を使えるなら、そちらを優先する

リンクを作る側は、「短いこと」よりも「後から検証できること」「長期的に維持できること」を優先すべきです。短縮 URL はリンク切れや管理不能化を起こしやすいため、印刷物や長期公開ページに残すと、将来の更新時に追跡できなくなることがあります。

2.6 QR コードなどもリンクである

メールや郵便物にある QR コードなどは、読み取るまで接続先 URL が見えにくいいため、フィッシングにも使われます。この手口は「クイッシング」とも呼ばれます。

メール本文に URL を文字列として書かず、画像内の QR コードに埋め込むことで、本文中の URL 文字列だけを調べる従来型のメール検査では検出されにくくなる点も悪用されています。現在は QR コードを解析する対策製品もありますが、すべてのメール環境で検査されるとは限りません。

- 読み取り後に表示される URL を確認してから開く
- ログインや支払いを求められたら、公式アプリや保存済みブックマークから開き直す
- メール本文だけでなく、PDF、画像、郵便物に印刷された QR コードにも注意を払うことが重要です。

また、広告などを作成する際、QR コードなどには適切にリンク先 URL などを明記するべきといえるでしょう。

2.7 ダウンロードファイルと拡張子の偽装

ファイル名や拡張子は自由に変更できます。最も危険な例として、見た目では文書に見えても、実際は実行ファイルであることがあります。また、Office 系の文書ファイルなどは単なる文書だけでなく、実行可能かつ攻撃的なマクロが含まれることもあります。

自身が危険であるファイルの例：

- document.pdf.exe
- image.jpg.scr
- invoice.zip

エクスペローラーなどで拡張子を表示しない設定にしていると、事故が起きやすくなります。Windows ではデフォルトで拡張子を表示しない設定になっていますが、これは不用意に拡張子を変更してしまい、ファイルが破損することを避ける意味であろうことは推測できるものの、セキュリティ的に望ましいとはいえません。

特に注意すべきファイル：

- .exe, .msi, .bat, .cmd, .ps1 などの実行ファイル・スクリプト
- .scr など、一見画像やスクリーンセーバーに見えるが実行される形式
- .zip, .7z, .rar などの圧縮ファイル（中身を隠せるため）
- .iso, .img などのディスクイメージ
- .lnk などのショートカットファイル
- .xslm, .xlam, .docm, .pptm などのマクロ付き Office ファイル

Excel や Word のマクロは、正しく使えば業務自動化に役立ちますが、悪用されるとファイルを開いた利用者の環境で処理を実行できます（なお、今日、本当にマクロでしか実現できない機能はそう多くありません。特に Excel などではワークシート関数で実現できるケースが多く、そもそもマクロを乱用することは避けるべきです）。「請求書」「見積書」「名簿」「申込一覧」など、業務上開きたくない名前前で送られることがあるため注意が必要です。

特に Excel ファイルでは、次の点を確認します。

- .xlsx ではなく .xslm や .xlam になっていないか
- 開いたときに「コンテンツの有効化」「マクロを有効にする」を求められていないか
- 送信者・入手元・用途が明確か
- そのファイルにマクロが必要な理由を説明できるか
- クラウド共有やメール添付で届いた場合、別経路で送信者に確認できるか

原則として、外部から届いた Office ファイルでマクロを有効にはいけません。団体内でマクロ付きファイルを使う場合も、作成者、配布場所、用途、更新履歴を明確にし、通常の文書や表計算ファイルとは区別して管理します。

個別のファイル形式をすべて覚えるよりも、入手元と実際の種類を確かめ、違和感があれば安全な経路へ戻るといった姿勢が重要です。具体的には、

- 拡張子を表示する設定にする
- 不審な添付やダウンロードは開かない
- マクロを有効にしない
- 圧縮ファイルは中身の拡張子まで確認する
- 公式サイトや信頼できる配布元から取得し直すことです。

2.8 ソフトウェア・アプリの導入元

ソフトウェアやアプリは、「何を入れるか」だけでなく、「どこから入れるか」が重要です。検索結果や広告からたどったダウンロードページ、SNS で紹介された配布ページ、非公式のミラーサイトなどには、偽アプリや改ざんされたインストーラーが紛れていることがあります。

特に注意すべき場面：

- 「無料版」「日本語版」「高速ダウンロード」などを強調する非公式サイト
- 検索結果の広告枠に表示されるダウンロードページ
- 本来有料のソフトウェアを無料で配布していると称するサイト
- スマートフォンで、公式ストア以外からアプリを入れるよう促す案内
- インストール時に、関係のない追加ソフトや拡張機能の導入を求める画面

基本は、公式サイト、OS やブラウザが提供する公式ストア、組織が指定した配布元から導入することです。また、インストール画面では「次へ」を連打せず、追加ソフト、ブラウザ拡張、通知許可、スタートアップ登録などが含まれていないか確認します。

「便利そうだから入れる」「有名そうだから入れる」ではなく、そのソフトウェアが本当に必要か、配布元が正規か、更新が継続されているかを確認する習慣が重要です。

2.9 パスワードの基本

パスワードは、いまでも多くのサービスで最初の防御線です。しかし、短いパスワード、使い回し、推測しやすい文字列は、漏洩や総当たり攻撃に弱くなります。

特に危険なのは、複数のサービスで同じパスワードを使うことです。あるサービスからパスワードが漏れると、攻撃者は同じメールアドレスとパスワードの組み合わせで、他のサービスにもログインを試みます。これは「パスワードリスト攻撃」と呼ばれます。

基本原則：

- サービスごとに異なるパスワードを使う
- 氏名、誕生日、団体名、電話番号など、推測されやすい情報を使わない
- 短い単語ではなく、十分に長いパスワードまたはパスフレーズを使う
- パスワードをチャット、メール、スプレッドシート、付箋などで共有・保管しない
- 可能であればパスワードマネージャーを使い、自分で多数のパスワードを暗記しようとする

「覚えやすい 1 つのパスワードをどこでも使う」ことは、利用者にとっては便利ですが、攻撃者にとっても便利です。現実的には、パスワードマネージャーでサービスごとに強いパスワードを生成・保存し、それに多要素認証を組み合わせるのが基本になります。

2.10 多要素認証 (2FA/MFA)

多要素認証とは、ログイン時にパスワードだけでなく、別の確認要素も求める仕組みです。2FA（二要素認証）は多要素認証の一種で、MFA (Multi-Factor Authentication) はより広い表現です。

認証要素は、大きく次の 3 種類に分けられます。

- 知っているもの：パスワード、PIN など
- 持っているもの：スマートフォン、認証アプリ、セキュリティキーなど
- 本人そのもの：指紋、顔などの生体情報

パスワードが漏洩しても、もう 1 つの要素がなければログインできないため、多要素認証は被害を大きく抑えます。特に、メール、クラウドストレージ、SNS、決済、サーバ管理、団体アカウントでは必須に近い対策です。

ただし、多要素認証にも強弱があります。一般に、SMS で届く確認コードよりも、認証アプリ、パスキー、セキュリティキーの方が安全性は高くなります。SMS は電話番号の乗っ取り、転送、端末紛失、通知のぞき見などの影響を受けるためです。

運用上の注意：

- 重要アカウントでは、可能な限り多要素認証を有効にする
- 認証アプリやパスキーを使える場合は、SMS だけに頼らない
- バックアップコードは安全な場所に保管し、通常のパスワードと同じ場所に置かない
- 機種変更時に認証手段を失わないよう、事前に移行手順を確認する
- 「認証コードを教えてください」という連絡には応じない

認証コードは、パスワードと同じく秘密情報です。相手が知人や運営を名乗っていても、コードを伝えてはいけません。

2.11 認証連携 (OAuth) の考え方

「Google でログイン」「Apple でサインイン」などは、認証連携の一種です。OAuth や OpenID Connect といった仕組みにより、利用者が新しいサービスへパスワードを直接渡さずに、本人確認や一部の情報・機能へのアクセスを認められるようにしています。

これはパスワードを増やさずに済む便利な仕組みですが、連携したサービスには、プロフィール情報、メールアドレス、カレンダー、クラウド上のファイルなど、許可した範囲のデータや機能へのアクセスが与えられることがあります。認証元のアカウントが乗っ取られれば、そのアカウントでログインしているサービスにも影響が及ぶ可能性があります。

連携を使うときは、誰が提供するサービスか、何へのアクセスを求めているか、その権限が目的に見合うかを考えます。過去に連携したサービスや使わなくなったアプリは、定期的に見直して不要な連携を解除します。「ログインを簡単にすること」と「広い権限を渡す」ことは別の判断です。

2.12 生体認証の考え方

生体認証は、指紋や顔など、利用者本人の身体的特徴を使って端末やサービスへのアクセスを許可する仕組みです。スマートフォンや PC のロック解除では、すでに一般的な方法になっています。

生体認証は便利ですが、「絶対に安全」という意味ではありません。多くの場合、生体情報そのものがサーバへ送られるのではなく、端末内の安全な領域で照合されます。そのため、端末のロック解除やパスキーの利用時には有効な手段です。

一方で、次の点に注意が必要です。

- 生体認証が使えない場合に備え、PIN やパスワードも適切に設定する
- 端末を他人に貸すときは、ロック解除後に見られる情報の範囲を意識する
- 顔認証や指紋認証を設定した端末を紛失した場合も、遠隔ロックやアカウント保護が必要になる
- 家族や近い関係者が端末を操作できる状態にしない

また、生体情報はパスワードのように簡単に変更できるものではありません。そのため、生体認証を扱う場合には細心の注意が必要といえるでしょう。

2.13 ログイン通知・利用履歴・連携アプリの確認

アカウントを守るうえでは、ログイン時の防御だけでなく、ログイン後の異常に気づくことも重要です。多要素認証を有効にしている、端末の紛失、セッションの乗っ取り、連携アプリの悪用などにより、意図しない利用が発生する可能性があります。

確認すべきもの：

- 身に覚えのないログイン通知がないか
- ログイン履歴に知らない地域、端末、ブラウザがないか
- 決済履歴、注文履歴、送信履歴に不審な操作がないか
- 外部アプリや外部サービスとの連携が増えていないか
- 使わなくなった端末や古いブラウザのログイン状態が残っていないか

不審なログインや操作を見つけた場合は、パスワード変更だけで終わらせず、すべての端末からログアウトする、連携アプリを解除する、多要素認証の設定を確認する、復旧用メールアドレスや電話番号が変更されていないか確認する、といった対応が必要です。

2.14 クラウド共有リンクの基本

クラウドストレージやオンライン文書の共有リンクは便利ですが、設定を誤ると情報漏洩につながります。特に「リンクを知っている全員が閲覧可」「編集可」「期限なし」の設定は、意図しない相手に広がったときに止めにくくなります。

共有前に確認する点：

- 本当にリンク共有が必要か、特定のアカウントだけに共有できないか
- 閲覧だけでよいのに編集可になっていないか
- ダウンロード、コピー、印刷を許可する必要があるか
- 共有期限やパスワードを設定できるか
- 共有した相手がさらに再共有できる設定になっていないか
- 文書内に非表示シート、コメント、変更履歴、別ページの個人情報が残っていないか

リンク共有は、「URL を知っている人なら誰でも入れる部屋」を作る操作になり得ます。個人情報や未公開資料を扱う場合は、原則としてアカウント指定の共有にし、作業が終わったら権限を外します。

2.15 端末管理の基本

多くの事故は、高度なハッキングではなく「更新していなかった」「ロックしていなかった」など基本から起きます。ソフトウェアアップデートやセキュリティアップデートは、既存の問題点を修正するものでもあるため、基本的に速やかにインストールするべきといえるでしょう。

基本的な事項として次のようなものが挙げられます。

- OS・ブラウザ・アプリなどは自動更新やセキュリティアップデートを ON にし、更新を先延ばしにしない
- 画面ロック（PIN・生体認証）を必須にし、自動ロック時間を短くする
- 不必要に管理者権限で作業をしない

特に私物端末を業務に使う場合は、これだけで事故の発生確率が大きく下がります。

2.16 スマートフォン特有の注意点

スマートフォンは、連絡、認証、決済、写真、位置情報、団体アカウントの操作などが集約された端末です。PC より気軽に使われますが、失ったときや乗っ取られたときの影響は非常に大きくなります。

特に注意すべき点：

- 公式ストア以外からアプリを入れない
- アプリの権限（連絡先、写真、位置情報、マイク、カメラ等）を必要最小限にする
- SMS やメッセージアプリのリンクからログイン・決済しない
- スマホ決済やクレジットカード連携には利用通知を設定する
- 端末の紛失に備え、画面ロック、遠隔ロック、端末を探す機能を有効にする
- 電話番号や SMS 認証だけに依存せず、認証アプリやパスキーも検討する

スマートフォンでは、画面が小さいため URL や送信元の確認が難になりやすく、通知から反射的に操作してしまうこともあります。重要な操作ほど、一度立ち止まり、公式アプリや保存済みブックマークから開き直すことが重要です。

2.17 キーロガーと入力情報の窃取

キーロガー (keylogger) とは、キーボードで入力した文字を記録・外部へ送信する不正ソフトウェアや機能のことです。パスワードだけでなく、検索語、メール本文、個人情報、認証コードまで盗まれるおそれがあります。PC では不審なソフトウェアや拡張機能、スマートフォンでは不正アプリが原因になることがあります。

スマートフォンのキーボードアプリは、入力した文字を変換・予測するため、当然ながら入力内容を扱います。サードパーティーのキーボードがすべて危険というわけではありませんが、便利な変換機能や AI 機能が入力内容を外部サーバーへ送信する場合もあります。そのアプリが何を収集・送信するかは、提供元、権限、プライバシーポリシー、利用中の機能によって異なります。

特に、パスワード、認証コード、決済情報、個人情報、未公開資料を入力する場面では、次を意識します。

- 必要性がなければ、OS 標準以外のキーボードを追加しない
- 導入する場合は、公式ストアから入れ、提供元と権限・プライバシーポリシーを確認する
- 入力内容をクラウドで処理する機能を使う場合は、送信してよい情報の範囲を判断する
- 重要な認証情報は、可能であればパスワードマネージャーやパスキーを使い、手入力する機会を減らす
- 身に覚えのないキーボード、ブラウザ拡張機能、アクセシビリティ権限を見つけた場合は、削除・無効化する前に状況を記録し、必要なら相談する

「キーボードは入力するための道具だから安全」とは限りません。入力内容を扱うアプリには、連絡先やカメラと同様に、必要最小限の権限だけを与えるという考え方が重要です。

2.18 バックアップとランサムウェアの基本

バックアップは、端末故障や誤削除だけでなく、ランサムウェアなどの被害から復旧するためにも重要です。ランサムウェアとは、端末やサーバ内のファイルを暗号化し、復旧と引き換えに金銭を要求する攻撃です。

基本的な考え方：

- 重要なデータを 1 か所だけに置かない
- クラウド同期をバックアップと混同しない
- 誤削除や暗号化が同期されても戻せるよう、履歴や世代管理を確認する
- 外付けドライブや別クラウドなど、通常作業環境から切り離された保存先を持つ
- 復元できるかを定期的に確認する

「保存しているつもり」でも、復元できなければバックアップとはいえません。特に団体の重要資料、申込データ、制作物、運用手順書などは、消えたときの影響を考え、保存場所と復旧方法を明確にしておきます。

2.19 フリー Wi-Fi と盗聴リスク

公共のフリー Wi-Fi では、通信内容を盗み見られる・偽アクセスポイントに接続させられる可能性があります。

避けるべき行為：

- 重要なログイン操作
- パスワード入力
- 個人情報・機密情報の送受信

やむを得ない場合は、

- VPN の利用
- 重要操作はモバイル回線に切り替える

などの対策を取るべきです。

現在は多くの Web サイトで HTTPS が使われているため、通信内容がそのまま丸見えになるとは限りません。ただし、偽アクセスポイント、フィッシング誘導、接続先情報の把握、端末側の共有設定などのリスクは残るため、重要操作を避けるという原則は変わりません。

2.20 偽セキュリティ警告・サポート詐欺

Web サイトを閲覧していると、突然「ウイルスに感染しています」「この PC はロックされました」「今すぐサポートに電話してください」などの警告画面が表示されることがあります。これは偽セキュリティ警告、またはサポート詐欺と呼ばれる手口です。

この種の画面では、大きな音、赤い警告、カウントダウン、全画面表示などにより利用者を焦らせ、画面上の電話番号へ連絡させようとします。しかし、画面が表示されただけで実際にウイルス感染しているとは限りません。むしろ、電話をかけたり、遠隔操作ソフトを入れたり、支払い情報を入力したりすることで被害が発生します。

遭遇したときの対応：

- 画面に表示された電話番号へ連絡しない
- 指示されたソフトウェアを入れない
- クレジットカード番号、認証コード、パスワードを入力しない
- ブラウザのタブやウィンドウを閉じる
- 閉じられない場合は、全画面表示を解除する、ブラウザや端末を終了する
- 組織端末や団体アカウントに関係する場合は、自己判断で処理せず共有する

重要なのは、「警告が出たからすぐ従う」のではなく、「警告そのものが攻撃かもしれない」と考えることです。

2.21 「急がせる」「不安にさせる」心理誘導

多くの攻撃は技術よりも心理を使います。

(例)

- 「至急対応してください」
- 「アカウントが凍結されます」
- 「今すぐ確認しないと問題が起きます」

重要なのは、急がされていると感じたら、一度止まることです。その上で、

- 公式サイトを自分で開き直す（メールなどのリンクから開かない）

- 送信者を別経路で確認するなどの対策を取るべきです。

2.22 不審なものを開いた・入力したかもしれないとき

どれだけ気をつけていても、誤って不審なリンクを開いたり、不審なフォームに情報を入力してしまう可能性はあります。不審なリンクを開いた際には速やかにタブを閉じることが基本的な対応といえるでしょう。また、不審なフォームに情報を入力してしまった場合は、必要に応じてパスワードの変更やクレジットカード決済の一時停止などを行うべきです。

組織に関する情報を入力した場合などは、自分だけで何とかしようとする被害が拡大します。この場合は、第 1 章 フェイルセーフとインシデント対応の方針に従い、情報を共有したうえで被害拡大を可能な限り防ぎます。

なお、Web サイトに設置されているフォームの類には、送信ボタンを押さなくともフォームへの入力内容を監視しているものもあります。このことを念頭に、常に最悪のケースを想定した対応を行うことがベストプラクティスといえます。

基本的な対応として、

- その場で追加操作をしない（ログイン継続や追加ダウンロードを止める）
- 関係者に速やかに共有する（原因究明よりも対応が優先です）
- 可能ならアカウントのパスワード変更・2FA 確認を行う

などが挙げられるでしょう。

また、組織でフィッシングメールなどを受信した際には、削除するだけでなく、管理者への報告を通じて組織全体に共有することで、他の人にも注意喚起を促せたり、また組織として必要な対応を取ることができたりします。

2.22.1 現実の事例：証券口座の乗っ取りと不正売買の急増

証券会社を装う偽サイト等でログイン情報を盗み、本人の証券口座へ侵入して、保有株式を売却したり別の銘柄を買ったりする不正取引が急増しました。パスワードを盗まれた後の被害を抑えるには、多要素認証、ログイン通知、取引履歴の確認も必要です。

この事例から学ぶ点：

- メールや SMS のリンクから証券口座へログインしない
- 公式サイトをブックマークし、多要素認証を有効にする
- 身に覚えのないログインや取引を見つけたら、すぐ証券会社へ連絡する

2.22.2 現実の事例：国税庁・日本郵便をかたるフィッシング

「未納税金がある」「財産を差し押さえる」「荷物を配達できなかった」「再配達の手続が必要」などと不安や焦りを起こさせ、偽サイトへ誘導するメールや SMS が確認されています。実在する組織名やロゴがあっても、本文中のリンクが正規とは限りません。

この事例から学ぶ点：

- 国税庁は、SMS やメールで納税や差押えを要求しない
- 配送状況は、メッセージ内のリンクではなく公式アプリや公式サイトから確認する
- 「本日中」「利用停止」「差押え」など、急がせる文言ほど立ち止まる

2.23 参考資料

- えきねっと「偽メール・偽サイトにご注意ください」^{*1}
- ITmedia NEWS「Google で『えきねっと』検索するとトップに偽サイト」^{*2}
- IPA「アプリのアクセス権限を確認しましょう」^{*3}
- IPA「偽セキュリティ警告（サポート詐欺）対策特集ページ」^{*4}
- IPA「情報セキュリティ 10 大脅威 2026」^{*5}
- 金融庁「インターネット取引サービスへの不正アクセス・不正取引による被害が急増しています」^{*6}
- 国税庁「不審なメールや電話にご注意ください」^{*7}
- 日本郵便「日本郵便を装った不審メール及び架空 Web サイトにご注意ください」^{*8}

2.24 確認

1. 検索結果の一番上にあるサイトが、必ず安全とは限らない理由は何か。
2. 自分が公式案内を作成するとき、短縮 URL の利用を避けるべき理由を 2 つ挙げよ。
3. QR コード型フィッシングで、接続先を開く前に確認すべきことは何か。
4. Web サイトで配布されていた .xls m ファイルをダウンロードして開くと、「マクロを有効にする」オプションが表示された。どう対応するのがよいか。
5. ソフトウェアやアプリを導入するとき、配布元として避けるべきものは何か。
6. パスワードの使い回しが危険な理由は何か。
7. 身に覚えのないログイン通知が届いたとき、確認すべきことは何か。
8. クラウド共有リンクで「リンクを知っている全員が編集可」という設定が危険な理由は何か。
9. スマートフォンでアプリ権限を確認すべき理由は何か。
10. 偽セキュリティ警告が表示されたとき、してはいけない行動を 2 つ挙げよ。
11. 官庁や銀行、配送会社などを名乗る緊急のメッセージを受け取ったとき、どのように確認するか。

^{*1} <https://www.eki-net.com/top/oshirase/security/>

^{*2} <https://www.itmedia.co.jp/news/articles/2212/14/news169.html>

^{*3} <https://www.ipa.go.jp/security/anshin/attention/2021/mgdayori20210914.html>

^{*4} <https://www.ipa.go.jp/security/anshin/measures/fakealert.html>

^{*5} <https://www.ipa.go.jp/security/10threats/10threats2026.html>

^{*6} https://www.fsa.go.jp/ordinary/chuui/chuui_phishing.html

^{*7} <https://www.nta.go.jp/information/attention/attention.htm>

^{*8} <https://www.post.japanpost.jp/notification/notice/fraud-mail.html>

第3章 個人情報

本章では、組織で活動するに当たって避けては通れない個人情報について扱います。個人情報の不適切な管理は取り返しのつかない問題に発展することも多いため、正確な理解が重要です。

3.1 個人情報とは何か

個人情報とは、「生存する特定の個人を識別できる情報」のことを指します。これには、氏名や住所、電話番号だけでなく、メールアドレス、顔写真、所属、履歴、組み合わせによって個人が特定できる情報も含まれます。また、個人情報保護法では「個人識別符号」を含む情報も個人情報に含まれます（詳細は後述）。

個人情報は小規模であっても適切な管理が必要です。1人分の個人情報でも、漏洩すれば当人に被害が発生し、団体の信用も失われます。

3.2 法令に基づく定義

この章の「個人情報」は、日本の個人情報保護法（個人情報の保護に関する法律）における用語を前提とします。実務では、一般的な感覚だけでなく、法令上の区分を理解して扱う必要があります。

3.2.1 用語の概要

- 個人情報：生存する個人に関する情報であって、(1) 氏名・生年月日等の記述等により特定の個人を識別できるもの（他の情報と容易に照合でき、結果として識別できるものを含む）、または(2) 個人識別符号を含むもの。
- 個人識別符号：それ自体で特定の個人を識別できる符号（例：旅券番号、健康保険の被保険者記号・番号等の番号類、指紋・顔等の身体特徴を一定の方法で変換した符号など）。個人識別符号が含まれる情報は、それだけで個人情報になります。
- 要配慮個人情報：差別や偏見等の不利益が生じないよう、取扱いに特に配慮を要する記述等が含まれる個人情報（例：人種、信条、社会的身分、病歴、犯罪の経歴、犯罪により害を被った事実など）。

3.2.2 「個人情報」から「個人データ」へ

個人情報保護法では、個人情報のうち、管理形態によりさらに用語が分かります。

- 個人情報データベース等：個人情報を含む情報の集合物で、検索できるように体系的に構成されたもの（コンピュータ上の名簿だけでなく、紙であっても索引等により容易に検索できる形で整理されていれば該当します）。
- 個人データ：個人情報データベース等を構成する個人情報。
- 保有個人データ：個人情報取扱事業者が、開示・訂正・利用停止・消去・第三者提供停止等の請求に応じ得る権限を有する個人データ（一定の例外あり）。

実務上は、次の関係で理解すると安全です。

- 個人情報（広い概念）
 - 個人データ（個人情報データベース等に記載して管理しているもの）
 - 保有個人データ（開示等の対象になり得るもの）

3.2.3 （参考）個人関連情報

個人情報に該当しない場合でも、個人に関する行動履歴等は「個人関連情報」として位置付けられ、提供の場面等で追加の制限が問題となることがあります（例：Cookie 等の端末識別子により収集される閲覧履歴、位置情報、購買履歴など）。

3.3 個人情報が発生する場面

特別な場面だけでなく、日常業務の中で個人情報は自然に発生します。

個人情報が発生する場面の例：

- フォームによる申込や応募
- 名簿や連絡先リスト
- メールのやり取り
- アンケート結果
- 写真や記録データ

アンケート等であっても、氏名等が含まれる、会員 ID 等とひも付く、回答内容が他の情報と容易に照合できる、といった場合は個人情報として扱う必要があります。これらは業務に必要な情報である一方で、管理方法を誤ると重大な問題になります。重要なのは、「どこで個人情報が発生しているか」を把握していることです。

3.4 個人情報の適切な使用

個人情報の事故は、「漏洩」だけでなく、「目的外の使用」でも発生します。

例えば、

- 連絡のために集めたメールアドレスを、別の案内に無断で使う
- 応募時に提出された情報を、別用途に転用する
- 名簿を、当初想定していなかった範囲に共有する

これらは漏洩していなくても、適切な取り扱いとはいえません。

重要なのは、

- 何の目的で収集したか
- その目的の範囲内でのみ使用しているか

を常に意識することです。目的外利用は、信頼を損なう大きな原因になります。

3.5 個人情報の扱いの原則

個人情報を扱うときは、目的・保存期間・共有範囲を意識することが重要です。

たとえばイベント申込で氏名やメールアドレスを受け取るのは、参加者へ連絡し、当日の受付を行うためです。この目的が終わった後まで同じ情報を持ち続ける理由があるか、改めて考える必要があります。「次の案内にも使えるかもしれない」という理由は、今回の申込の目的とは別です。

同様に、運営に関わる人全員が申込情報を見られる必要があるとは限りません。受付を担当する人、連絡を担当する人、会計処理を担当する人では、必要な情報や期間が異なることがあります。

該当の目的のもと、保存期間と共有範囲を最小限にすることが原則といえます。

3.6 保存期間と削除

個人情報は、「持っているだけ」でリスクになります。不要になった個人情報を長期間保存していることは、事故の確率を上げる行為です。

そのため、

- 何の目的で収集したか
- いつまで保存する必要があるか
- 不要になったらどう削除するか

をあらかじめ決めておく必要があります。

削除のルールが決まっていないと、「とりあえず残しておく」という状態が続き、データが蓄積されていきます。

3.7 アクセス権限の設計

個人情報は、必要な人だけがアクセスできる状態にする必要があります。「共有フォルダに置いておけば便利」という発想は、事故の原因になります。

重要なのは、

- 誰が閲覧できるか
- 誰が編集できるか
- 誰がダウンロードできるか

を明確に分けることです。

アクセスできる人が多いほど、事故の確率は高くなります。

- 面識のない相手を To/CC に並べて一斉送信する（メールアドレス漏洩）
- 「リンクを知っている人は全員閲覧可」で名簿を共有する

などは不適切な取り扱いの代表例です。

3.8 削除と匿名化

集計や実績のためにデータを残したい場合でも、個人情報のまま残す必要がないことが多いです。なお、ここでいう「匿名化」は一般的な意味であり、法令上の「匿名加工情報」等（要件のある概念）と同義ではありません。匿名化が不十分で再識別できる場合、個人情報として扱う必要があります。

匿名化の例：

- 氏名・メールは削除し、参加人数・属性の集計だけ残す
- 連絡先一覧は更新し、退会者の情報は消す

「残す価値がある情報」と「持っているだけでリスクとなる情報」を分け、前者のみを保存することはよい方針といえるでしょう。

3.8.1 現実の事例：上智大学、全正規生 13,949 人分の情報を誤送信（2024 年）

上智大学では、海外の協定校へ留学プログラム参加者のリストを送る際に、学生情報を含むマスターデータのシートを削除しないままファイルを送信しました。その結果、全正規生 13,949 人分の学生情報が外部へ渡る事案となりました。

この事例の中心は、メールの操作ミスだけではありません。本来は限られた参加者の情報だけを扱えばよい場面で、より広範な個人情報を含むマスターデータが同じファイルに残っていたことにあります。

個人情報の管理では、「元データを持っているか」だけでなく、そこから作られた集計表・共有用ファイル・ダウンロード済みのコピーに、どの情報が残っているかを考える必要があります。目的に必要な情報だけを取り出した共有用データを作るとは、誤送信時の影響を小さくすることにもつながります。メール送信時の添付ファイル確認については 第 5 章 メール を参照してください。

3.9 事故が起きたとき

個人情報の漏洩・誤送信・誤共有が疑われる場合は、第 1 章 フェイルセーフとインシデント対応の原則に従い、

- 被害拡大を止める（共有停止/権限剥奪/公開取り下げ）
- 速やかに関係者に共有すること
- 速やかに関係者に共有すること

また、個人情報の漏えい等では、内容や規模によって、本人への通知や個人情報保護委員会への報告が必要になる場合があります。細かい手続を暗記する必要はありませんが、**「個人情報の事故は内部対応だけで完結しないことがある」**と理解しておくことが重要です。

3.10 参考資料

- e-Gov 法令検索「個人情報の保護に関する法律（平成十五年法律第五十七号）」^{*1}
- 個人情報保護委員会「個人情報の保護に関する法律についてのガイドライン（通則編）」^{*2}
- 個人情報保護委員会「漏えい等の対応とお役立ち資料」^{*3}
- Security NEXT「全正規生約 1.4 万人分の個人情報、海外協定校への送信ファイルに残存」^{*4}

3.11 確認

1. 氏名とメールアドレスは個人情報か。
2. 個人情報の目的外利用に当たる例を挙げよ。
3. 個人情報はなぜ「持っているだけ」でリスクになるのか、述べよ。

^{*1} <https://laws.e-gov.go.jp/document?lawid=415AC0000000057>

^{*2} https://www.ppc.go.jp/personalinfo/legal/guidelines_tsusoku/

^{*3} <https://www.ppc.go.jp/personalinfo/legal/leakAction/>

^{*4} <https://www.security-next.com/159300>

第4章 著作権

文芸団体であるからは、著作権に関する理解は必要不可欠でしょう。しかし、著作権に関する項目には決して単純明快とはいえないものがあります。なんとなくの運用は大きなトラブルの温床となります。本章では、基本的な著作権の考え方を扱います。

4.1 著作権とは何か

著作権とは、著作物を創作した人に与えられる権利です。文章、画像、写真、音声、動画、デザインなど、多くのものが著作物に該当します。重要なのは、「公開されているから自由に使ってよい」というわけではないことです。インターネット上にある画像や文章も、原則として著作権で保護されています。著作権は申請や登録をしなくても、創作と同時に自動的に発生します。

4.2 利用してよいかを考える起点

作品や素材を使う前に問うべきなのは、「インターネットで見つけたか」ではなく、「これは誰の表現で、どのような条件で自分たちに預けられているか」です。自分や団体が作ったもの、作者から明確に許可されたもの、利用条件が示されたものでは、扱い方が異なります。

一方で、作者や利用条件が分からない素材には、利用を正当化する根拠がありません。使いたい気持ちが強いほど、「たぶん大丈夫」と解釈せず、別の素材を選ぶか、作者に尋ねるという選択が重要です。

4.3 作者の意図と二次利用

著作権には、「お金に関わる権利」だけでなく、著作者人格権と呼ばれる、作者の意向や名誉に関わる権利も含まれます。

たとえば、依頼を受けて寄せられた作品を短く編集する、作者名を外して紹介する、イベント用の原稿を後から広報に転用するといった行為は、作品そのものの価値や作者との関係を変え得ます。一度受け取った作品でも、最初に想定されていなかった使い方まで自動的に許されるわけではありません。

4.4 ライセンス

フリー素材・オープンな文章でも、利用条件（ライセンス）は様々です。作者名の表示、加工の可否、利用できる場面などが定められていることがあります。

代表的な仕組みに、Creative Commons(CC) ライセンスがあります。これは著作権を手放すものではなく、作者が「この条件なら再利用してよい」とあらかじめ示す仕組みです。記号には次の意味があります。

- BY（表示）：作者名や作品名など、指定されたクレジットを示す
- NC（非営利）：営利目的での利用を認めない
- ND（改変禁止）：作品を改変した形で利用しない
- SA（継承）：改変して公開する場合は、同じ条件の CC ライセンスを引き継ぐ

たとえば CC BY は表示を条件に再利用・改変を認めるライセンスであり、CC BY-NC-ND は表示をしたうえで非営利かつ改変しない利用に限るライセンスです。トリミング、色調補正、翻訳、作品の一部だけを切り出すことも、条件によっては「改変」に当たるため注意が必要です。

CC0 は CC ライセンスとは別の、著作権を可能な限り放棄してパブリックドメインへ提供するための仕組みです。ただし、CC や CC0 の表示があっても、作品内に写る人物の肖像や、別の作者の素材まで自由に使えるとは限りません。ライセンスの表示が、実際に使いたい作品・部分に適用されているかを確認します。

4.5 スクリーンショットは「画面の引用」ではない

画面を画像として切り取っても、そこに写る文章・画像・画面デザインの権利が消えるわけではありません。有料記事や会員向け情報を広げてしまうこともあります。また、名前、ID、通知、連絡先などの写り込みは、著作権とは別に個人情報の問題を生みます（第 6 章 SNS・公開情報・テストデータ管理参照）。

「画面を見せること」が本当に必要なのか、それとも自分の言葉で説明したり、元のページへリンクしたりすれば足りるのかを考えることが大切です。

4.6 同意は「後で使える権利」ではない

写真・作品・寄稿などの著作物は、イベント当日の利用、SNS での紹介、Web への再掲、長期アーカイブなど、時間とともに使いたい場面が増えがちです。いったんある場面での使用に同意を得たからといって、別の場面にもその著作物を使用してよいことにはなりません。当初と異なる使い方をしたいときは、改めて作者・本人の意思を確認める必要があります。

同じ種類の利用が継続的に起こる団体では、担当者の記憶や個別のやり取りだけに頼らず、利用に関する基準をあらかじめ定めることが推奨されます。たとえば、写真をどこまで公開するか、応募作品を冊子・Web・SNS・アーカイブのどこで扱うか、作者名をどのように表示するかを共有しておけば、参加者も自分が同意する内容を判断しやすくなります。

応募・投稿を受け付ける場合は応募規約、寄稿・制作を依頼する場合や対価が発生する場合は契約書など、場面に応じた文書を用いることが有効です。文書には、利用媒体、期間、改変の可否、クレジット表示、対価、第三者の権利に関する扱いなどを、実際の活動に即して定めるべきでしょう。高額な取引、広い再利用、権利の移転を伴う場合は、安易なひな形の流用ではなく、専門家への相談も検討すべきです。

4.7 正しい引用

「出典を書いたから引用になる」とは限りません。引用は、自分の説明・批評のために他者の表現を必要な範囲で用いる考え方であり、本文と引用部分の関係、必要性、区別、分量などを総合して考えられます。

他者の作品そのものを見せることが目的になっている場合は、引用として扱えるとは限りません。自分の言葉で要約できないか、許可を得られないかを含めて、表現の仕方を選ぶことが大切です。

4.7.1 現実の事例：ニュース記事をまとめサイトへ無断転載（2026 年）

2026 年 3 月、新聞社等の取材記事をニュースまとめサイトへ無断転載したとして、サイト運営に関係した男性が著作権法違反の疑いで書類送検されました。

事実そのものは著作物でない場合がありますが、取材して構成・表現された記事本文をコピーしてよいわけではありません。

この事例から学ぶ点：

- 出典やリンクを付けても、記事本文の転載が自動的に適法になるわけではない
- 見出しや事実を参考に自分で説明することと、記事の表現をコピーすることを区別する
- 引用に当たるかは、目的、主従関係、必要性、区別などを総合して判断する

4.7.2 現実の事例：AI で映画をカラー化した海賊版の販売

白黒映画を生成 AI でカラー化し、権利者の許諾なく複製・販売した事件で、2026 年 1 月、大阪地方裁判所が有罪判決を言い渡しました。

AI で色を付けたり加工したりしても、元の映画の著作権が消えるわけではありません。

この事例から学ぶ点：

- AI による加工は、元作品を自由に利用できる根拠にならない
- 古い作品、白黒作品、入手困難な作品にも著作権が残っている場合がある
- 生成 AI の利用規約だけでなく、入力・加工する素材と出力物の権利を確認する

4.8 参考資料

- 文化庁「著作権を学ぶ（資料・講習会）」^{*1}
- クリエイティブ・コモンズ・ジャパン「クリエイティブ・コモンズ・ライセンスとは」^{*2}
- 一般社団法人コンピュータソフトウェア著作権協会 (ACCS)「取材記事を無断転載、男性を送致」^{*3}
- 一般社団法人コンテンツ海外流通促進機構 (CODA)「生成 AI でカラー化した海賊版 DVD 販売事件」^{*4}

4.9 確認

1. 「ネットで公開されている画像」は自由に使えるか。
2. フリー素材でも注意すべき点は何か。
3. スクリーンショット公開で起きやすい事故は何か。
4. 後から用途を変えて再利用する場合、なぜ問題が起きやすいか。
5. ある句集の批評を行うために、句集の全句を引用した。これは適切か。

^{*1} <https://www.bunka.go.jp/seisaku/chosakuken/seidokaisetsu/>

^{*2} <https://creativecommons.jp/licenses/>

^{*3} <https://www2.accsjp.or.jp/criminal/2025/1158.php>

^{*4} <https://coda-cj.jp/news/2635/>

第 5 章 メール

今日、メールを使わないで生活することはほとんど不可能でしょう。しかし、メールの仕様や正しい使い方を理解している人はあまり多くありません。本章では、必ず理解しておくべきメールの基本的なトピックについて扱います。

5.1 To/CC/BCC の正しい使い分け

メール事故で最も多いのは、宛先設定の誤りです。原則は次の通りです。

- 互いに面識のない複数人へ送る場合、To や CC に並べてはいけない
- この行為は、メールアドレスという個人情報の漏洩にあたる
- 一斉連絡は原則 BCC を使用する

実際の事例：

- T 俳句会では、新人会メール句会の幹事が全員分のアドレスを To に入れて句稿や句会報を送信してしまっている。
- T 大俳句会で、行事の案内を送信する際、誤って参加者全員のアドレスを To に入れてしまった。To や CC を使ってよいのは、受信者同士が互いのアドレスを知っていて問題ない関係である場合のみです。

なお、人数が多い一斉連絡では、BCC だけに頼ると誤操作や履歴管理の問題が起きやすくなります。大量配信や継続的な案内では、メーリングリストや配信システムなど、目的に合った仕組みを使うことも検討します。

5.2 送信前に立ち止まる観点

メールは「送った瞬間に取り返しがつかない」代表例です。ここでは送信操作を覚えるのではなく、受信者の立場から見て何が起り得るかを想像する習慣を身につけます。

送信ボタンを押す前に、以下を確認すべきです。

- 宛先 (To/CC/BCC) は正しいか
- 面識のない複数人が含まれる場合、適切に BCC を使用しているか
- 誤ったファイルを添付していないか
- 本文に個人情報・機密情報を書き過ぎていないか
- 返信・転送時の引用部分に不要な情報が残っていないか

5.3 メールアドレスと表示名の違い

メールソフトに表示される「名前」は、送信者が自由に設定できます。そのため、表示名だけでは送信元の正当性を判断できません。

(例)

- 表示名：official@example.com
- 実際のアドレス：scam@example.com

このような偽装は古典的ですが、現在も頻繁に使われています。

重要なのは、表示名ではなく、実際のメールアドレスを見ることです。たとえば Gmail や Outlook では、名前 <メールアドレス>の形式で表示されます。

5.4 メールエイリアスと + による区別

メールエイリアスは、一つの受信先に対して、複数のメールアドレスを使えるようにする機能です。個人用・問い合わせ用などでアドレスを分けても、同じメールボックスで受信できる場合があります。ただし、作成方法・送信元として使えるか・外部サービスで受け付けられるかは、メールサービスや団体の設定によって異なります。

一部のメールサービスでは、基本のアドレスの @ より前に + と任意の文字列を付けたアドレスを使えます。例えば、`organization@example.com` を使っている人が `organization+event@example.com` をイベント申込用として使うと、対応するサービスでは同じ受信箱へ届きます。このように申込先ごとに表記を変えると、どの登録先から届いたメールかを見分ける手がかりになります。

- + 付きアドレスに対応していないサービスや、入力を受け付けられない Web フォームもあるため、常に使えるとは限りません。
- +event の部分は秘密情報ではありません。メールが転送・漏えいすれば他者にも見えるため、パスワード、認証コード、個人情報を入れてはいけません。
- エイリアスは、別アカウントを作成するものでも、迷惑メール対策の保証でも、多要素認証の代替でもありません。重要な連絡先や団体アカウントでは、利用可否と管理方法をあらかじめ確認しておくべきです。

5.5 なりすまし・フィッシングの確認観点

次の要素が揃うほど危険です。

- 返信を急がせる/不安を煽る（「至急」「凍結」「支払い」等）
- リンク先が短縮 URL、またはドメインが微妙に違う
- 返信先 (Reply-To) が不自然
- 添付ファイルを開かせようとする

迷ったら、メール内リンクを開かず、公式サイトを自分で開く、送信者に別経路で確認する、などの対応を取るべきです。

5.6 フィッシングの典型

多くの不審なメールは文面に不自然なところがありますが、そうではなくとも安全なメールであるとは限りません。次はフィッシングメールの典型例です。

- アカウント停止や凍結を装い、ログインを急がせる
 - 請求書・領収書・配送通知を装い、添付やリンクを開かせる
 - 「パスワード期限切れ」「2FA 設定が必要」などを口実にする
- 「急がせる」「不安を煽る」「リンクを踏ませる」といった要素が揃うほど危険といえます。

5.7 不審メールを判断するときの順序

迷ったときは、メールの文面がもっともらしいかではなく、急がせる仕掛けから距離を取れるかで判断します。次の順序は、そのための考え方です。

1. 焦って操作や返信をしない
2. メール内リンクを開かず、公式サイトを自分で開く
3. 団体内の既知の連絡先へ、別経路で確認する（電話や別チャット等）

4. 被害が疑われる場合は、第 1 章 フェイルセーフとインシデント対応 に従い速やかに共有する

5.8 BCCで受信したときの注意点

自分がBCCに入っているメールに対して「全員に返信」を行うと、他の受信者の存在を露呈させてしまうことがあります。BCCは、受信者同士を見えなくするための仕組みです。返信時には、送信先を慎重に確認する必要があります。

5.9 添付ファイルとリンクの扱い

メールは攻撃の入口になりやすいので、以下のような点に特に注意すべきです。

- 添付は必要最小限にする（可能ならクラウド共有＋権限定）
- 不審な添付ファイルは開かない（拡張子・送信者・文脈を確認）
- メール内リンクは安易に開かない（公式サイトを自分で開く）

5.9.1 実際の事例：上智大学、全正規生 13,949 人分の情報を誤添付（2024 年）

第 3 章 個人情報で扱われた上智大学の事例は、表示中のシートだけを確認し、非表示部分や元データを含むファイル全体を確認しなかった典型例です。画面上で見えていない情報も、ファイル内に残っていれば受信者が閲覧できる可能性があります。

メールの添付事故は、宛先を間違えたときだけに起こるものではありません。添付ファイルでは別シート・非表示の内容・コメント・変更履歴など、画面に見えていない情報も受信者へ渡り得ます。必要な項目だけを新しいファイルへ書き出すという考え方が、元の台帳を加工して送るより安全な場合があります。

5.10 メールを介したサイバー攻撃と回避策

メールは、サイバー攻撃の最も一般的な入口です。代表的な手法には次のようなものがあります。

- フィッシング（偽サイトへ誘導）
- マルウェア付き添付ファイル
- パスワード変更を装うリンク
- 緊急性を装って判断を急がせる文面

回避策として重要なのは、

- 不審なリンクをクリックしない
- 添付ファイルを安易に開かない
- メールからではなく、公式サイトを直接開く
- 内容に違和感があれば、別経路で確認することです。

5.11 誤送信・誤添付をしてしまったとき

報告と対応が重要です。第 1 章 フェイルセーフとインシデント対応 の原則に従い、

- 速やかに関係者へ共有する
- 可能なら共有リンクの無効化/権限剥奪などで被害拡大を止める
- 受信者へ訂正連絡が必要か検討する（個人情報の有無で判断）

を優先します。

5.12 迷惑メールフォルダの扱い

迷惑メールフォルダは「不要なメール置き場」ではありません。正規のメールが誤判定で入ることがあります。重要な連絡が届かない原因になるため、

- 定期的に迷惑メールフォルダを確認する
- 正常なメールを「迷惑メールではない」と設定することが必要です。

5.13 共有メール/代表アドレス運用の注意

団体では代表アドレスを複数人で扱うことがあります。このときは、

- 担当者が変わっても追跡できるよう、履歴を残す
 - 返信担当が分かる運用（ラベル/チケット/スレッド管理）を決める
 - 個人の私物アカウントに転送して放置しない
- などに配慮すべきです。

5.14 確認

1. 面識のない複数人への一斉送信で使うべき宛先種別はどれか。
2. 受信したメールの送信者欄に `info@example.jp` `<info@example.com>` と表示された。送信元メールアドレスを答えよ。また、このメールは安全と考えられるか否か。それはなぜか。
3. フィッシングが疑われるとき、最も安全な確認方法は何か。
4. 誤送信に気づいたら最初に何をすべきか。

第 6 章 SNS・公開情報・テストデータ管理

全員がときにメディアとなり得る現代において、普段からどのような情報を発信するかに意識を向けることは重要です。また、SNS などにおいては、匿名性に起因する、時に不可解なトラブルも起こりやすいといえます。こうしたリスクをなくすことは不可能ですが、可能な限りトラブルを避けるためには、一般的な知識や過去の事例に学ぶことは効果的といえるでしょう。本章では、SNS・公開情報・テストデータの管理など、情報発信にかかわる項目について扱います。

6.1 公開情報と内部情報の境界

SNS や Web サイトへの投稿は、手軽で即時性がありますが、一度公開した情報は、完全に消去することができません。該当の投稿やページを削除しても、

- スクリーンショット
- キャッシュ
- 転載

などによって残り続ける可能性があります。そのため、投稿前に不適切な内容や機密情報が含まれていないか確認する必要があります。

6.2 情報区分の 3 段階

一般に、情報は以下の 3 種類に区分できます。

- 公開：Web/SNS に出してよい
- 内部：メンバー内では共有するが、外部公開しない
- 機密：漏れると被害が大きい（個人情報、認証情報、未発表企画、審査情報、内部連絡先など）
迷ったら「内部」以上として扱い、公開しないのが安全です。

6.3 画像や動画のリスク

SNS では、投稿そのものだけでなく、写真やスクリーンショット、動画への映り込みなどが事故の原因になります。たとえば、スクリーンショットを用いた投稿で画面の一部だけを見せたつもりでも、URL やメールアドレス、ファイル名、ディレクトリ構造など、公開するべきでない情報が含まれてしまうことがあります。画面共有やスクリーンショットは、想像以上に多くの情報を含みます。

また、ライブ配信などで、意図しない情報を公開してしまうことがあります。画面全体の共有を避け、特定のウィンドウに限って共有する、必要以外のウィンドウを開かないなどの対策を取るべきです。

6.4 画像のメタデータや映り込み、特定可能性

写真には撮影日時・機種・位置情報などが含まれることがあります。公開する可能性のある写真は、

- 公開する意図のない位置情報やメタデータが入っていないか
 - 背景に住所・表札・名札・会員名簿等が写っていないか
- を確認するべきです。

また、写真には、想像以上に多くの情報が含まれています。メタデータなどの位置情報を削除していても、画像そのものに含まれる手がかりから、撮影場所や時間帯、所属先、生活圏が推測される場合があります。GeoGuessr のような位置推定ゲームで使われる観察手法は、悪用されれば個人や団体の特

定にもつながります。たとえば、道路標識、電柱、店名、看板、建物の形、植生、車線、舗装、山や海の見え方、影の向きなどは、撮影場所を絞り込む材料になります。

また、撮影日時や投稿時刻が分かる場合、天気、太陽の位置や影の向き・角度から、かなりの精度をもって撮影位置を推測されることもあります。これは単独では決定的な情報でなくても、背景の地形、建物、イベント日程、SNS の過去投稿などと組み合わせると、特定可能性が高まります。特定されることを避けたい場合にはこうした要素も考慮する必要があります。

さらに、目、眼鏡、スマートフォン画面、窓、鏡、金属、車体、モニターなどの反射にも注意が必要です。反射には、撮影者の顔、周囲の人物、部屋の様子、PC 画面、書類、名札、位置を示す看板などが映り込むことがあります。

写真や動画を公開する前には、次の観点でも確認します。

- メタデータを削除しても、画像本体から場所や時間が推測されないか
- 背景の看板、窓の外、交通機関、建物、地形などが場所の手がかりにならないか
- 影の向き、空の明るさ、イベント状況などから撮影日時が絞り込まれないか
- 目、眼鏡、鏡、窓、金属、画面などの反射に情報が写っていないか
- 過去投稿や他人の投稿と組み合わせると、個人や団体が特定されないか

6.4.1 現実の事例：入社書類を Instagram へ投稿（2026 年）

2026 年 4 月、三菱電機の子会社へ入社した新入社員が、社員番号、所属部署、氏名、機密保持誓約書等が写った入社書類を Instagram へ投稿したと報じられました。

「入社したことを共有する」投稿でも、背景や書類に内部情報・個人情報が含まれれば情報漏えいになります。

この事例から学ぶ点：

- 社員証、入館証、誓約書、配属通知などを SNS へ掲載しない
- 自分の情報であっても、社員番号や内部様式は組織の管理情報になり得る
- 親しい友人限定や短時間公開でも、保存・転載される前提で判断する
- 新任者には、業務開始前に具体的な写真例を使った SNS 教育を行う

6.4.2 現実の事例：西日本シティ銀行の BeReal 投稿（2026 年）

2026 年 4 月、西日本シティ銀行の職員が営業店の執務室内で撮影した動画や画像を BeReal へ投稿し、別の SNS で拡散されました。

銀行の公表では、映り込んだホワイトボードから顧客 7 人の氏名が外部へ漏えいしました。

この事例から学ぶ点：

- 職場で撮影するだけで、書類、画面、名札、ホワイトボード等が写り込む
- 24 時間で消える投稿や限定公開でも、スクリーンショットや転載は防げない
- 「撮影してよい場所」と「投稿してよい内容」は別に判断する
- 個人のスマートフォンを職場へ持ち込む場合も、撮影ルールを明確にする

6.5 ブレースホルダーとテストデータの事故

テストや例示のために作成したデータが、そのまま公開される事故は非常に多く発生します。

事故例：

- 仮のメールアドレスや個人名が実在していた

- サンプル画像やサンプル文書に個人情報が残っていた
- 不謹慎な内容を含んだテストデータを削除し忘れ、意図せず公開されてしまった

重要なのは、テストデータであっても、実在の個人情報を使わないこと、テストデータであっても、外部に流出して困る内容を書かないことです。

ダミーデータとしてのメールアドレスや URL などは、example.com（または example.net や example.org）を使用することが推奨されています。ただし、テストでメールを受信する必要がある場合は使用できません。

6.6 テストデータの作り方（安全を優先する）

テストデータは、公開するつもりがなくとも、何らかの形で公開される可能性があるものと想定して作成すべきです。

原則：

- 実在の個人情報を使わない（氏名・メール・住所・電話など）
 - 実在団体や実在人物を想起させる内容を書かない
 - 外部に出ても支障のない内容にする（不謹慎な内容、内部事情、金額、審査結果などを混ぜない）必要なら、
 - 生成したダミーデータ
 - ドメインは example.com 系
- を使うべきであり、実データは使ってはいけません。

6.6.1 現実の事例：「セシウムさん」テロップ誤放送（2011 年）

東海テレビの番組で、リハーサル用に作られた不適切な当選者名「セシウムさん」が誤って放送されました。テスト表示が本番系統に残り、確認と切替えの仕組みも十分に機能しなかったことで、単なる入力ミスでは済まない重大な問題になりました。

この事例から学ぶ点：

- テストデータも公開される前提で、差別的・攻撃的な文言や実在情報を使わない
- 本番とテストを見た目と仕組みの両方で分ける
- 公開直前の確認者と確認対象を明確にする
- 身内では通じる言葉でも、公開されたときに誰を傷つけるかを想像する

6.7 投稿前に考えるべきこと

SNS 投稿は個人の注意力だけに任せると、ミスが起きやすくなります。重要なのは「投稿できるか」ではなく、「公開後に誰がどのように受け取るか」を複数の視点で考えることです。そのため、

- 投稿前に内容を見直す
- 別の人が確認する仕組みを持つ
- 公開して問題ない情報かをチェックする

といった確認手順を設けることで、事故を防ぐことができます。

6.8 なりすまし・偽アカウント・DM 詐欺

SNS では、団体や関係者を装った「なりすまし」が発生します。

なりすまし例：

- 団体公式に似せたアカウントが、DM でリンクを送ってくる
- 「当選」「参加登録」「支払い」などを口実に、偽フォームへ誘導する
- 公式発表前の情報を引き出そうとする

こうした脅威に対応するため、以下の項目を理解しておくことが重要です。

基本原則：

- DM や返信で届いたリンクは、安易に踏まない
- アカウント名/ID の微妙な違いに注意する
- 公式サイトや既知の連絡先などの別経路で確認する

被害が疑われる場合は、第 1 章 フェイルセーフとインシデント対応 の初動（止血・共有）を優先します。

6.9 炎上と誤解を防ぐ（文脈とスクリーンショット）

SNS は文脈が切り取られやすく、意図しない解釈が頻繁に発生します。

- 内部向けの話題や、身内向け前提の表現は避ける
- 断定表現、攻撃的表現は避ける
- スクリーンショットで拡散される前提で書く

「消したら増える」というネットミームもありますが、「削除すればよい」という考え方は炎上の前には一切通用しません。

問題が起きたときに、投稿者や受け手を責める反論、事実確認をしないままの言い訳、説明のない削除を重ねると、投稿内容そのものとは別に「組織は問題を理解していない」という不信を招くことがあります。まず公開を止め、何が起きたかと誰に影響があるかを確認し、根拠のある説明と必要な謝罪・是正を行うことが重要です。第 1 章 フェイルセーフとインシデント対応 の「止血→共有→記録」という考え方が、SNS でも役立ちます。

6.9.1 現実の事例：ほっかほっか亭のエイプリルフル投稿（2025 年）

2025 年 4 月 1 日、ほっかほっか亭の公式 X が、全国の店舗でライスの販売を停止するという内容をエイプリルフル企画として投稿しました。当時は米価高騰が社会的な関心事となっており、投稿を事実と受け取って不安を感じた人や、状況への配慮が足りないと感じた人から批判が集まりました。同社は後に謝罪しています。

この事例から分かるのは、エイプリルフルのように「冗談だと分かるはず」と思える企画でも、公式アカウントから発信されれば、利用者は事業上の正式な告知として受け取ることがあるという点です。投稿者の意図だけでなく、その時の社会状況、受け手の不安、団体・企業に期待される役割まで含めて考えなければなりません。

6.10 参考資料

- coki「三菱電機子会社・新卒社員が入社書類を SNS 公開」^{*1}
- 西日本シティ銀行公式 X「お詫びとお知らせ」^{*2}

^{*1} <https://coki.jp/article/column/74121/>

^{*2} https://x.com/ncbank_official/status/2049708655252299789

- ITmedia NEWS「銀行員が支店内で BeReal 投稿」*3
- 東海テレビ放送「岩手県産米プレゼント当選者テロップ誤放送」検証報告*4
- J-CAST ニュース「『#エイプリルフール』つきでも炎上 ほっかほっか亭『ライス販売停止』投稿で謝罪」*5

6.11 確認

1. 投稿を削除しても情報が残り得る理由は何か.
2. スクリーンショット投稿で注意すべき点は何か.
3. テストデータで実在情報を避けるべき理由は何か.

*3 <https://www.itmedia.co.jp/news/articles/2604/30/news096.html>

*4 <https://www.tokai-tv.com/press/pdf/2011/110830.pdf>

*5 <https://www.j-cast.com/2025/04/01503029.html>

第7章 生成 AI と情報管理

生成 AI はこの数年で飛躍的な進化を遂げ、新たな社会インフラの様相さえ呈しています。しかし、生成 AI の特性を理解しない使用者による不適切な利用事例が後を絶ちません。生成 AI も包丁などと同じく、便利な道具ではありますが、不適切な使用をすると危険をもたらし可能性もあります。本章では、現時点における生成 AI に関する基本的・一般的な考え方について扱います。

7.1 生成 AI は「便利な道具」だが、入力と出力に事故がある

生成 AI は、もはや単に業務効率を上げるのみならず、日常の一部となり、また多くの可能性を生み出すツールとなりました。一方で、生成 AI に伴う事故も多く発生しています。事故は、入力に関する事故と出力に関する事故に大別できます。

- 入力：出してはいけない情報を入力してしまう（個人情報、機密、秘密情報等）
- 出力：それっぽい誤情報（ハルシネーション）や、権利・機密を侵害する内容が混ざる

7.2 入れてはいけない情報の原則

生成 AI に入力した内容は、

- どこに保存されるか
- 学習に使われるか
- 誰が運用上アクセスできるか

がサービスごとに異なります。そのため、まずサービス仕様に依存しない保守的な原則として、次を入力しません。

- 個人情報（氏名、メール、住所、電話、顔写真、応募情報、名簿等）
- 認証情報（パスワード、トークン、API キー、秘密鍵、バックアップコード等）
- 非公開の内部資料（未発表企画、議事録のドラフト、審査情報、内部連絡網など）
- 契約・支払・法的トラブル等の機密情報

判断に迷う場合は入力しません。

実際にどのサービスを業務で使ってよいか、入力内容が学習に使われるか、保存期間や管理者権限がどうなっているかは、サービスや契約形態によって変わります。本資料では個別サービス名を覚えるのではなく、使う前に仕様と団体のルールを確認するという姿勢を重視します。

7.3 匿名化と要約

個人情報を含む文章の体裁を整えたい場合など、AI を用いたいが、内容をそのまま入力することは不適切な場合があります。その場合は、

- 個人を特定できる情報を削除・置換する（例：A さん、B 社、X イベント）
- 数字や固有名詞を丸める・一般化する
- 目的に必要な部分だけに削る

といった匿名化・最小化を行ってから入力するとよいでしょう。

7.4 生成結果は「提案」であり、責任は人が負う

生成 AI は、もっともらしく誤った内容を出力することがあります。

重要な原則：

- 最終判断・最終責任は人が持つ
- AI 出力物を公開・提出・送信・納品した場合、その内容に対する責任は使用者や組織が負う
- 「AI が生成した」「AI の回答を信じた」という説明は、法的責任や対外的な説明責任を免れる理由にはならない
- 事実関係（日時、金額、URL、引用元、規約、手順）は必ず一次情報で確認する

特に、対外発信（SNS/公式サイト/一斉メール）は、第 6 章 SNS・公開情報・テストデータ管理と同様に確認を挟みます。

たとえば、AI が作成した文章に誤情報、名誉毀損、著作権侵害、個人情報の不適切な掲載、不当な案内などが含まれていた場合でも、実際に公開・利用した人や組織が責任を問われる可能性があります。AI 出力物は、自分が作成した資料と同じように確認してから使います。

7.4.1 現実の事例：チャットボットの誤案内と事業者の責任

Air Canada の Web サイト上のチャットボットが、弔事運賃の申請方法について誤った案内を行い、利用者が不利益を受けた事案があります。

カナダの紛争解決機関は、チャットボットも事業者の Web サイトの一部であり、掲載情報の正確性について事業者が責任を負うと判断しました。

この事例から学ぶ点：

- AI の出力を「AI が言ったこと」として責任から切り離すことはできない
- 公開情報や利用者への案内は、人が根拠を確認する
- 誤りを発見した際に、訂正、利用停止、影響確認ができる体制を持つ

7.5 著作権・引用・「もっともらしい文章」の危険

生成 AI の出力は、

- 既存文章に似た表現が混ざる
- 出典不明のまま断定する

ことがあります。

対策：

- 生成結果をそのまま貼り付ける前提にせず、草案として用い、文脈に合わせて書き直す
- 引用が必要なら出典を必ず確認し、出典が取れないなら使わない
- 画像やロゴ、作品の利用は 第 4 章 著作権 の判断手順に従う

7.6 コーディングに使うときの注意（秘密情報と脆弱性）

現在のコーディング AI は、単にチャット欄へコードを貼って相談するだけでなく、IDE や CLI 上でリポジトリを読み、ファイルを編集し、テストやコマンドを実行する「コーディングエージェント」として使われることがあります。そのため、注意点も「何を入力するか」だけでは足りません。AI がどのファイルを読めるか、どの操作を実行できるか、どの差分を採用するかを管理する必要があります。

起きやすい事故：

- リポジトリ全体を AI に読ませ、秘密情報や内部資料まで文脈に含めてしまう
- .env、秘密鍵、認証トークン、本番ログ、個人情報を AI に読ませてしまう
- AI が生成したコードに脆弱性（XSS、SQL インジェクション、認可漏れ等）が混ざる

- AI が不要なファイルを書き換える、設定を壊す、テストを弱める
- コマンド実行により、外部送信、依存関係の追加、データ削除などの副作用が起きる

基本方針：

- AI に読ませる必要のないファイルは、AI 向けの除外設定で読ませない
- ツールが対応している場合は、`.aignore` などの AI 向け ignore ファイルを使う
- `.env`、秘密鍵、認証トークン、本番ログ、個人情報、顧客データ、バックアップを除外対象にする
- `.gitignore` は Git 管理から外す設定であり、AI が読む範囲の制御とは別に考える
- コード相談は最小限の再現例にする（実コードや本番データを貼りすぎない）
- 秘密情報や個人情報を含むログは提示しない
- AI が作った差分は、採用前に人がレビューする
- テストを追加・実行し、失敗したテストを削除してごまかしていないか確認する
- コマンド実行、依存関係の追加、外部通信、データ削除は承認制にする
- セキュリティ上の原則（第 9 章 サーバ・コーディング関係のセキュリティ）に照らして確認する

特に、秘密情報は「AI に貼らない」だけでは不十分です。エージェントが自動でファイルを読む環境では、秘密情報をリポジトリ内に置かないこと、置かざるを得ない場合は読み取り対象から除外すること、実行前にアクセス範囲を確認することが重要です。

代表的な除外対象：

- `.env`、`.env.local`、`.env.production`
- 秘密鍵、証明書、トークン、バックアップコード
- 本番ログ、エラーログ、アクセスログ
- データベースダンプ、バックアップ、CSV 出力
- 個人情報、顧客情報、応募者情報、名簿
- 契約書、請求書、未公開資料

AI 向け ignore ファイルの名前や対応範囲はツールによって異なります。`.aignore`、`.cursorignore` などを使うものもあれば、設定画面や権限設定で管理するものもあります。重要なのはファイル名を暗記することではなく、AI に読ませない情報を明示的に管理することです。

7.7 参考資料

- Civil Resolution Tribunal of British Columbia, *Moffatt v. Air Canada*, 2024 BCCRT 149^{*1}

7.8 確認

1. AI に入力してはいけない情報を 3 つ挙げよ。
2. 生成結果をそのまま信じると危険な理由は何か。
3. AI の出力を使用したことに伴う損害などの責任は、一般に誰に帰せられるか。
4. コーディング AI に秘密情報を読ませないための具体策は何か。

^{*1} <https://decisions.civilresolutionbc.ca/crt/crtd/en/item/525448/index.do>

第 8 章 ファイル管理・バックアップ・ログ・Git

ファイル管理が適切に行われていない場合、重要な情報が散逸したり、錯綜したりするなどして問題やミスの温床となります。部屋を整理整頓するように、データも正しく整理しておくことが重要です。本章では、これらの基礎的な知識を概観するとともに、主に技術者向けに、ファイル管理に関する技術についても扱います。

8.1 フォルダ構造と命名規則

ファイル管理の事故の多くは、特別なトラブルではなく、「どこに何があるかわからない」「どれが最新版かわからない」といった状態から始まります。

(例)

- 「関西俳句バトル 2100-投句一覧-最新版-000101」
- 「関西俳句バトル 2100-投句一覧-最終版 (校正済)」
- 「関西俳句バトル 2100-投句一覧-最終確定版」
- 「最終更新-関西俳句バトル 2100-投句一覧-本当の最終版」
- 「関西俳句バトル 2100-投句一覧-確定版 2」
- 「関西俳句バトル 2100-投句一覧-本当の最終版-final」

これは個人の整理能力で解決できる問題ではなく、フォルダ構造や命名規則が決まっていないことが原因です。

このような事例を減らすには、統一されたルールを設けることが効果的です。また、ルールには、

- 日付 (YYYY-MM-DD) をファイル名に入れる
 - 「最新版」ではなく「版本号/日付」で管理する
- などの対策を含めるとよいでしょう。

8.2 共有フォルダの権限

共有ドライブや共有フォルダは便利ですが、不適切な権限設計は誤編集や誤削除の温床となります。基本方針：

- 編集できる人は最小限にする
- 閲覧専用の共有を用意する
- むやみに「リンクを知っている人は全員」の共有設定にしない (第 2 章 情報リテラシ概論)

8.3 バックアップとバージョン履歴

ファイルは、削除や上書きによって簡単に失われます。これを防ぐためには、バックアップとバージョン履歴の考え方が重要です。多くのクラウドサービスには、

- 過去の状態に戻す機能
- 削除したファイルを復元する機能

が備わっています。また、それらの機能が「あることを知っている」だけでなく、実際に使える状態にしておくことも重要です。

8.4 ログ（記録）を残す発想

事故対応で重要なのは「誰が悪いか」より「何が起きたか」です。

ログの価値：

- いつ何が変わったか分かる
- どこまで影響したか切り分けられる
- 再発防止策（仕組み）に繋げられる

ログが残る環境を選ぶことや、権限共有でログを壊さないことが重要です。

8.5 Git の概念

ファイルを管理する上で、変更履歴をたどったり、また必要に応じて過去のバージョンに戻したりしたくなることがあります。また、複数人で作業をする場合に、最新版のファイルを毎回やり取りしたり、同時に編集したりすることは、取り違えや意図しない上書きの可能性などを考慮したときに望ましい状態とはいえません。こうした問題を解決するシステムとして、特にコーディングの分野で多く使われているのが Git です。

Git はプログラマだけの道具ではありません。「履歴を残しながら変更を管理する」という考え方そのものが重要です。Git の考え方には、次のような利点があります。

- いつ、どのような変更をしたか記録できる
- 過去の状態に戻すことができる
- 複数人で同時に作業できる
- 変更点（差分）が明確にわかる

これは文書や設定ファイルの管理にもそのまま応用できます。

8.6 Git で管理してよいもの・いけないもの

Git は強力ですが、何でも入れてよいわけではありません。

基本方針：

- 入れてよい：公開しても問題ない文書、設定（Secrets を除く）、手順書、テンプレ等
- 入れてはいけない：個人情報、秘密情報（Secrets）、未公開の内部資料（公開前提でないもの）

「非公開リポジトリだから大丈夫」という判断は危険です。誤公開・権限設定ミス・スクリーンショット共有等で流出する遠因となるからです。

8.7 Git の基本用語

Git では、「変更をどこに置き、誰が確認できる状態か」を区別して考えます。

8.7.1 置き場所

- リポジトリ (repository / repo)：ファイルと変更履歴をまとめて保管する単位です。
- ローカル (local)：自分の PC 上にあるリポジトリです。作業中の変更は、まずここにあります。
- リモート (remote)：GitHub などにある共有用のリポジトリです。チームで共通に見る置き場です。

8.7.2 変更を履歴にする

- 差分 (diff) : どのファイルの、どの部分を変更したかを示す表示です。
- コミット (commit) : 関連する変更を「この目的で行った変更」として履歴に確定する単位です。
- コミットメッセージ : その変更の理由や内容を示す短い説明です。後から履歴を読むための手がかりになります。

コミット前の変更は、まだ自分の作業途中です。コミットして初めて、変更を戻したり他者に説明したりしやすくなります。

8.7.3 本流と作業の分岐

- main (メイン) : 通常、正式な最新版として扱う本流のブランチ名です。
- ブランチ (branch) : 本流を壊さずに変更を試すための作業上の分岐です。
- マージ (merge) : ブランチで行った変更を本流へ統合することです。

8.7.4 チームへ共有する操作

リモート — clone / pull —> ローカル — push —> リモート

- クローン (clone) : 共有リポジトリを自分の PC へ最初に複製し、作業を始めることです。
- プル (pull) : 共有場所の最新の変更を、自分の PC へ取り込むことです。
- プッシュ (push) : 自分の PC でコミットした変更を、共有場所へ送ることです。
- プルリクエスト (PR / Pull Request) : ブランチの変更を本流へ取り込む前に、差分を見てもらいレビューを依頼する仕組みです。

8.7.5 Git に入れないもの

- .gitignore : 一時ファイルや秘密情報を誤って追加しないために、Git の管理対象から除外する設定です。

ただし、すでにコミットしたファイルを .gitignore に書いても、履歴から消えるわけではありません。 .env などの秘密情報を入れないための予防策であり、秘密情報を安全に保管する仕組みそのものではない点に注意します。

8.8 よくある誤解

- 誤解 : プッシュしたら、自動的に本流 (main 等) が書き換わる
 - 現実 : 通常は「PR でレビュー→マージ」まで進まない限り、本流は変わらない
- 誤解 : コミット = 公開
 - 現実 : コミットはローカルでもできる。公開 (共有) はプッシュしたときに起きる
- 誤解 : プル (pull) とプルリクエスト (PR) は同じ
 - 現実 : プルは「リモート→ローカルに取り込む操作」、PR は「変更を取り込む前のレビュー依頼」

8.9 共同作業を安全にする見取り図

Git の共同作業は、個々のコマンドを暗記するよりも、変更が「誰に見えているか」「どこまで戻せるか」を意識して捉えると事故が減ります。次の流れは、その状態を保つための代表例です。

1. クローンして作業を開始する（最初の 1 回）
2. 作業前にプルして最新にする（他人の変更を取り込む）
3. ブランチを切って変更する（本流を直接いじらない）
4. 変更をコミットして履歴に残す
5. プッシュしてリモートへ共有する
6. PR でレビューし、問題なければマージして本流に取り込む
「レビューできる」「いつでも戻せる」ことが、Git の価値です。

8.10 Git の特長

クラウド文書にも履歴機能はありますが、Git は次が強いです。

- 変更理由をコミット単位で残しやすい
 - 複数人の変更が衝突したときに、差分として整理できる
 - 誤変更を「どの変更で起きたか」まで追しやすい
- 文章や規約、設定、テンプレのように「少しずつ改訂されるもの」と相性が良いです。

8.11 共同作業で起きやすい事故と回避

事故は技術ではなく、運用の曖昧さから起きやすいです。

- 本流（main 等）に直接反映してしまう
 - 回避：ブランチ + PR + レビューを基本にする
- 変更が混ざって、何をしたのか分からなくなる
 - 回避：コミットを小さくする（1 コミット=1 目的）

8.12 公開/非公開の確認

Git の事故で多いのは「うっかり公開」です。

- リポジトリが公開か非公開かを最初に確認する
- 公開してよい情報だけを入れる（非公開でも同様）
- 外部公開が必要な場合は、公開対象を選別した別リポジトリに分離する

8.13 Git 運用で特に注意する点（実務）

Git は強い一方で、次の事故が頻発します。

- 認証情報（トークン、API キー、パスワード）をコミットしてしまう
- うっかり公開リポジトリにしてしまう
- 「一度入った履歴」は完全消去が難しいことを知らない

基本方針：

- “公開してよい情報” しか Git に置かない
- 秘密情報 (Secrets) は Git に入れない（環境変数や専用保管へ）
- 公開/非公開の設定を最初に確認する

もし誤って秘密情報 (Secrets) を入れてしまった場合は、削除コミットだけでは足りないことが多いので、第 1 章 フェイルセーフとインシデント対応 の手順で速やかに共有し、無効化/再発行を優先します。

8.14 .env ファイルも Git に入れない

.env は、API キー、データベース接続情報、パスワードなどを環境変数として読み込むためによく使われるファイルです。

しかし、ファイル名が .env であるだけで暗号化やアクセス制御が行われるわけではありません。

基本方針：

- .env は原則として .gitignore に指定する
- 共有するときは、チャットやメールに貼らず、専用の Secrets 管理機能を使う
- リポジトリには値を空欄やダミーにした .env.example だけを置く
- 誤ってコミットした場合は、削除だけでなく Secrets を無効化・再発行する

8.15 「秘密情報 (Secrets) を入れてしまった」時の考え方

Secrets の事故では、次の誤解が多いです。

- 誤解：「消せば大丈夫」

現実：

- Git は履歴が残るため、単にファイルを消しても“過去の履歴”に残り得る
そのため優先順位は、まず止血です。

1. 当該 Secrets を無効化・再発行する
2. 影響範囲（どこで使っていたか）を確認する
3. 必要なら履歴の除去を検討する（技術者が対応）

ここでも 第 1 章 フェイルセーフとインシデント対応 の原則（止血→共有→記録）が基本になります。

8.16 実務で意識すべきポイント

ファイル管理において重要なのは、高度な技術ではなく、次の点です。

- 保存場所と命名規則を決める
- クラウドの履歴・復元機能を活用する
- ログが残る環境を利用する
- 「履歴を残す」という Git の思想を取り入れる

これらを実践することで、誤削除や誤編集による事故の影響を大きく減らすことができます。

8.16.1 現実の事例：マネーフォワードの GitHub 不正アクセス（2026 年）

2026 年、マネーフォワードグループが開発・システム管理に利用する GitHub へ第三者が不正アクセスし、ソースコードが閲覧・コピーされました。同社の調査では、サービス更新作業中に個人情報を含むファイルが本来の管理手順から外れ、誤って GitHub 上に保管されていたことも公表されています。

本番データベースや金融機関との連携に必要な認証情報への侵害は確認されませんでしたが、安全確認のため銀行口座連携機能が一時停止され、GitHub 上の個人情報について対象者への連絡が行われました。

この事例から学ぶ点：

- 非公開リポジトリでも、不正アクセスや権限侵害は起こり得る
- ソースコード以外の作業用ファイルを、リポジトリへ混入させない
- 個人情報や Secrets を検出する仕組みを、コミット前と GitHub 側の両方に設ける
- 事故時には、直接侵害された範囲だけでなく、関連機能を一時停止して安全確認する場合がある
- 本番データベースの漏えいと、リポジトリ内ファイルの漏えいを区別して説明する

8.17 参考資料

- マネーフォワード「GitHub への不正アクセス発生および銀行口座連携機能の一時停止」^{*1}
- マネーフォワード「GitHub への不正アクセスに関する調査結果のご報告（第四報）」^{*2}

8.18 確認

1. 中～大規模プロジェクトにおける、適切なファイル命名規則の例を示せ。
2. 秘密情報を誤って Git にコミットした場合、なぜ削除コミットだけでは不十分になり得るか。

^{*1} <https://support.me.moneyforward.com/hc/ja/articles/57504390625305>

^{*2} <https://corp.moneyforward.com/news/info/20260623-mf-press-1/>

第 9 章 サーバ・コーディング関係のセキュリティ

本章では、主に技術者向けに、基本的な開発におけるセキュリティに関する知識をまとめます。パイプコーディングが普及してきた今日、古典的な脆弱性を見落としたまま公開してしまう危険性に改めて目を向けるべきでしょう。なお、本章に挙げたものは代表的なものの一部に過ぎません。OWASP Top 10 などの最新の情報に注意を払うこと、また必要に応じてセキュリティレビューを行う AI などを活用することなども現代においては重要かつ効果的といえるでしょう。

9.1 クライアントとサーバの役割分離

Web アプリや Web サイトでは、ブラウザで動く部分（クライアント）と、データや処理を担う部分（サーバ）に役割が分かれます。この部分を誤解すると、見た目では制限しているつもりでも、実際には誰でも突破できる設計になりがちです。

重要な原則は次の通りです。

- クライアント側の制限は、あくまで表示上の制限にすぎない（利用者は開発者ツール等で挙動を変えられる）
- 権限チェックや重要な判断はサーバ側で行う
- サーバは「届いた入力信用しない」という前提で設計する

9.2 認証 (Authentication) と認可 (Authorization)

Web の事故で多いのは「ログインはあるが権限チェックがない」状態です。

- 認証：誰か（本人確認）
- 認可：その人がその操作をしてよいか（権限）

認証があっても、認可がないと、

- 他人のデータが見える
- 管理操作ができる

といった重大事故になります。

9.3 API キー・トークン・環境変数

API キーやトークン、パスワードなどの「秘密情報」は、漏れると即座に不正利用される可能性があります。特に多い事故は、「ソースコードや設定ファイルに埋め込む」「公開場所に置く」ことです。基本方針は次の通りです。

- Secrets はコードに書かず、環境変数や専用の Secrets 管理機能で扱う
- 権限は最小限（漏れても被害が限定されるようにする）
- 定期的にローテーション（無効化・再発行）できる運用にする
- 公開してよい情報と絶対に公開してはいけない情報を明確に区別する

9.4 .env と環境変数の注意

`.env` は環境変数を読み込むための一般的な方法ですが、Secrets を安全に保管する金庫ではありません。平文ファイルなので、Git への混入、バックアップからの流出、共有時の誤送信などが起こります。

注意点：

- 開発用の `.env` は Git に入れず、`.env.example` には変数名とダミー値だけを書く
- 本番では、ホスティングサービスや OS 等の Secrets 管理機能を使う
- 環境ごとに別の Secrets を使い、権限を最小化する
- フロントエンド用の環境変数は、ビルド後の JavaScript に含まれて利用者から見える場合がある
つまり、環境変数にただで秘密になるわけではなく、誰が読める場所に置かれるかまで確認する必要があります。

9.5 Secrets 漏洩時の現実的な優先順位

漏れた Secrets は「漏れたかもしれない」の段階でも被害が出ます。

優先順位：

1. そのキー/トークンを無効化・再発行する（技術的に止血）
2. 影響範囲を把握する（ログ確認）
3. 再発防止（なぜ混入したかの仕組み改善）

個人の注意力ではなく、仕組み（Secrets 管理、確認、CI）で防ぎます。

9.6 代表的な攻撃 1：XSS（クロスサイトスクリプティング）

XSS は、攻撃者が入力欄などを通してスクリプト (JavaScript) を混入させ、それが他の利用者の画面で実行されることで被害が起きる攻撃です。

起きると何が困るか：

- 利用者の操作を乗っ取る
- セッション情報を盗む
- 偽画面を表示して入力を誘導する

対策の原則：

- 出力時エスケープ（表示する前に危険な文字を無害化）
- HTML を文字列結合で生成しない（安全なテンプレート機構等を用いる）
- ユーザー入力をそのまま HTML として解釈しない
- 必要に応じて Content Security Policy (CSP) を設定する
「入力を禁止する」ではなく、「入力が増えても安全に表示する」設計が重要です。

9.7 代表的な攻撃 2：IDOR（権限不備による他人データ参照）

IDOR (Insecure Direct Object Reference) は、たとえば URL や API の ID を変えるだけで他人のデータが見えてしまう事故です。

原因：

- 「その ID のデータをその人が扱ってよいか」の認可チェックがない

対策：

- すべてのデータ参照/更新に認可を入れる
- “フロントでボタンを隠した”だけで満足しない

9.8 RLS(Row-Level Security) によるデータベース側の防御

RLS は、データベースの行ごとに「誰が読めるか、変更できるか」を制限する仕組みです。たとえば、利用者が自分の投稿だけを参照・更新できるようにすることで、アプリ側の認可漏れがあっても被害を抑えられる場合があります。

重要な考え方：

- アプリケーション側の認可と RLS を組み合わせる（多層防御）
- SELECT だけでなく、INSERT、UPDATE、DELETE も確認する
- 一般利用者、管理者、未ログイン利用者など、役割ごとにテストする
- 管理者権限、テーブル所有者、BYPASSRLS 等は RLS を迂回する場合がある
- RLS を有効にただけで満足せず、ポリシーと実際の接続権限を確認する

RLS は強力ですが、認証、API 側の認可、最小権限、監査ログの代わりにはなりません。「URL の ID を変えても他人の行は取得できないか」を自動テストに含めることが重要です。

9.8.1 現実の事例：UTopia のアクセス制御不備（2026 年）

2026 年 4 月に公開された大学生向けマッチングサービス「UTopia」は、データベースのアクセス制御設定の不備により、一部利用者のメールアドレス、表示名、性別、年齢、大学・高校名、プロフィール画像が外部から閲覧可能だったと公式に公表しました。公式発表では、本人確認で提出されたマイナンバーカード画像は漏えい対象に含まれていないとされています。

外部の技術検証では、バックエンドに使われた Supabase で RLS が適切に設定されていなかった可能性や、RLS を迂回できる強い権限を持つ `service_role` キーがフロントエンドに含まれていた可能性も指摘されています。

この事例から学ぶ点：

- 画面上でログインを要求するだけでは、データへのアクセス制御にならない
- Supabase の公開スキーマでは、対象テーブルすべてについて RLS とポリシーを確認する
- SELECT だけでなく、INSERT、UPDATE、DELETE を未ログイン・一般利用者・管理者の各権限でテストする
- `service_role` キーなど RLS を迂回する Secrets を、ブラウザやアプリへ配布しない
- 本人確認書類など、漏えい時の影響が大きい情報を安易に収集・保存しない

9.9 代表的な攻撃 3：CSRF（クロスサイトリクエストフォージェリ）

CSRF は、ログイン済みの利用者に気付かれない形で、別サイト経由で意図しない操作を実行させる攻撃です。

起きると何が困るか：

- 本人の意思と無関係に、設定変更・投稿・送金等が実行される

対策の原則：

- CSRF トークンの導入
- GET で状態変更をしない
- 重要操作は POST 等にしたうえで、確認手順を設ける
- Cookie 属性（SameSite 等）の適切な設定
- 重要操作は再認証を要求するなど、被害を抑える設計

9.10 代表的な攻撃 4：SQL インジェクション

SQL インジェクションは、入力欄や URL パラメータなどを通して、本来の意図と異なるデータベース操作を実行させる攻撃です。

起きると何が困るか：

- 他人のデータが読まれる
- データが改ざん・削除される
- 管理者権限の操作につながる

対策の原則：

- SQL を文字列結合で組み立てない
- プレースホルダー/プリペアドステートメントを使う
- データベース権限を最小限にする
- エラー内容を外部に詳しく出さない

「入力値をチェックする」だけで防ぐのではなく、危険な SQL として解釈されない仕組みにすることが重要です。

9.11 ファイルアップロード/添付機能の注意

ファイル機能は事故が発生しやすい領域です。

注意点：

- 拡張子だけで判断しない（中身を信用しない）
- 実行可能な場所に保存しない（Web から実行されない配置）
- ウイルススキャン等を挟む（可能なら）
- 公開範囲（誰がダウンロードできるか）を必ず制御する

9.12 バリデーションと認可

Web の多くの脆弱性は、「想定外の入力」を受け取ったときに発生します。そのため入力に対して、次の二段が必要です。

- バリデーション：形式・範囲・型が正しいか（例：数字のはずが文字列）
- 認可 (Authorization)：その人がその操作をしてよいか（権限チェック）

特に危険なのは「フロント側で弾いているから大丈夫」という思い込みです。攻撃者はフロントを経由せずに直接リクエストを送れます。

9.13 ログと監査

セキュリティは防ぐだけでなく、気づくことも重要です。

最低限ログに残したいもの：

- ログイン/ログアウト
- 重要操作（権限変更、データ削除、公開設定変更）
- Secrets の発行/更新/失効

ただしログ自体に個人情報や Secrets を出さないよう注意します。

9.14 環境分離（本番/検証/開発）

本番データを開発環境にコピーすると、テストデータ事故（第 6 章 SNS・公開情報・テストデータ管理）や漏洩に直結します。

原則：

- 本番の個人情報を開発/検証に持ち込まない
- 必要なら匿名化/マスキングしてから使う
- 本番と同じ Secrets を使い回さない

9.15 依存パッケージとサプライチェーン

JavaScript の開発では多数の外部ライブラリを利用します。便利な一方で、脆弱なライブラリや乗っ取られたパッケージが混入すると、自分のコードに問題がなくても被害を受けます。

対策の原則：

- 依存関係を最小化する（必要なものだけ）
- 脆弱性アラートの確認と更新
- ロックファイルの管理（意図しない更新を防ぐ）
- 不明な作者・不自然な人気急増など、危険信号に注意する
- 依存追加や大きな更新は、差分・公開元・更新履歴を確認してから行う
- CI/CD 上の Secrets は最小権限にし、漏れい時にすぐ無効化できるようにする

9.15.1 現実の事例：npm パッケージ汚染と Shai-Hulud 系マルウェア（2025 年～2026 年）

2025 年以降、npm では、正規のパッケージやメンテナンスアカウントが乗っ取られ、悪意あるコードを含むバージョンが公開されるサプライチェーン攻撃が相次いでいます。代表例として、Shai-Hulud と呼ばれる自己増殖型のマルウェアや、その派生とされる Mini Shai-Hulud/Miasma 系の攻撃が報告されています。

これらの攻撃では、利用者が「有名なパッケージだから安全」と考えて通常どおり `npm install` や CI の自動ビルドを実行しただけで、開発環境や CI/CD 環境から GitHub トークン、npm トークン、クラウド API キー、SSH キーなどの Secrets が読み取られるおそれがありました。さらに、盗まれた認証情報を使って別のパッケージやリポジトリに悪意あるコードを広げる動きも確認されています。

この事例から学ぶ点：

- 有名パッケージや正規アカウントから配布されていても、汚染されることがある
- 依存パッケージのインストールは、単なるファイル取得ではなく、コード実行になり得る
- 開発端末や CI/CD には強い権限の Secrets を置きがちなので、被害が連鎖しやすい
- 依存関係を自動更新している場合、悪意あるバージョンを即座に取り込む危険がある
- 被害が疑われる場合は、パッケージを戻すだけでなく、トークンや API キーの無効化・再発行が必要になる

対策として、ロックファイルを使って意図しない更新を抑える、更新差分を確認する、CI ではインストールスクリプトや外部通信を必要最小限にする、Secrets を最小権限・短寿命にする、異常な依存更新や Secrets アクセスを検知できるようにする、といった多層防御が必要です。

9.16 脆弱性対応で問われる判断

脆弱性はゼロにできません。重要なのは、個別の製品手順を暗記することではなく、見つかったときに影響と緊急度を見誤らず、速く確実に動けることです。

最低限の運用：

- 依存関係の更新/アラートを定期的に確認する
- 影響範囲（どの機能/環境）を切り分ける
- 修正→テスト→リリース→監視、の流れを決めておく

事故対応は 第 1 章 フェイルセーフとインシデント対応 の考え方に統合します。

9.17 実務で意識すべきポイント

この章の内容は、具体的なコードの暗記ではなく、次の原則を運用に落とすことが目的です。

- 重要判断（権限チェック）はサーバで行う
- Secrets はコードに書かず、権限最小で管理する
- XSS は「出力時エスケープ」、CSRF は「トークンと Cookie 設定」が基本
- SQL インジェクションは「文字列結合しない」「プレースホルダーを使う」が基本
- 入力は必ずバリデーションし、操作は必ず認可する
- 依存ライブラリを点検し、更新・監視の仕組みを持つ
- 依存パッケージの汚染を前提に、CI/CD と Secrets を最小権限で設計する

高度な攻撃に備える前に、まず「基本原則を外さない設計」を徹底することが最も効果的です。

9.18 参考資料

- PostgreSQL Documentation, *Row Security Policies*^{*1}
- UTopia 公式 X「発生した事象」^{*2}
- UTopia 公式サイト^{*3}
- Supabase Documentation「Row Level Security」^{*4}
- IA Insight Lab「東大生限定マッチングアプリ UTopia の情報漏洩」^{*5}
- TechRadar「Mini Shai-Halud hackers publish over 600 compromised npm packages」^{*6}
- Tom's Hardware「Shai-Hulud malware campaign dubbed 'the largest and most dangerous npm supply-chain compromise in history」^{*7}

^{*1} <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>

^{*2} https://x.com/UTopia_matching/status/2049745638821408930

^{*3} <https://utopia.tokyo/>

^{*4} <https://supabase.com/docs/guides/database/postgres/row-level-security>

^{*5} <https://note.com/hirotsuchida/n/na2f33030bd9f>

^{*6} <https://www.techradar.com/pro/security/mini-shai-halud-hackers-publish-over-600-compromised-npm-packages-developers-warned-to-be-on-their-guard>

^{*7} <https://www.tomshardware.com/tech-industry/cyber-security/shai-hulud-malware-campaign-dubbed-the-largest-and-most-dangerous-npm-supply-chain-compromise-in-history-hundreds-of-javas-crypt-packages-affected>

- ITPro 「Shai-Hulud malware is back with a vengeance」^{*8}

9.19 確認

1. クライアント側の制限だけでは不十分な理由を述べよ。
2. 認証と認可の違いは何か。
3. `.env` に書けば Secrets が安全になる、とは限らない理由は何か。
4. RLS の役割と、RLS だけでは不十分な理由は何か。
5. XSS 対策の原則は何か。
6. CSRF 対策の原則は何か。
7. SQL インジェクション対策の原則は何か。
8. Secrets 漏洩時の最優先対応は何か。
9. 本番データを開発環境にコピーすることが危険な理由は何か。
10. npm パッケージ汚染のようなサプライチェーン攻撃で、パッケージを戻すだけでは不十分な理由
は何か。

^{*8} <https://www.itpro.com/security/cyber-attacks/shai-hulud-malware-is-back-with-a-vengeance-and-hit-more-than-19-000-github-repositories-so-far-heres-what-developers-need-to-know>

確認の解答

第1章 フェイルセーフとインシデント対応

1. 人のミスや機器の故障が起きても、危険や被害が小さい側へ動くように、あらかじめ仕組みを設計する考え方です。たとえばエレベーターには、ロープの異常などで過度に落下した場合にガイドレールをつかんで停止させる安全装置があるため、一部の部品が故障しても落下被害を防ぐ方向に働きます。
2. まず被害拡大を止め、関係者へ共有し、分かっている事実を記録します。原因が判明するまで報告を待つと、不正利用や誤公開が続き、対応の機会を失うため危険です。不明な点は不明なまま共有します。
3. 強い叱責だけでは、ミスや異常を隠し、早期報告を妨げるおそれがあります。たとえば飲食店でアレルギー表示の誤りを見つけた従業員が、強く責められることを恐れて報告しなければ、訂正や提供停止の機会を失い、利用者の健康被害につながり得ます。再発防止では、個人を責めるだけでなく、確認手順や表示の仕組みを整えます。

第2章 情報リテラシ概論

1. 広告、検索順位の操作、偽サイト、侵害された正規サイトなどが上位に表示されることがあるためです。検索順位は正規性や安全性を保証しません。
2. 行き先のドメインを利用者が確認しにくいこと、短縮サービスの停止・乗っ取り・リンク切れに依存することなどです。
3. 読み取った後に表示される URL のドメイン、HTTPS の有無、公式サイトと一致するかを確認します。重要な操作は QR コードから直接行わず、公式サイトを自分で開きます。
4. 配布元と業務上の必要性を確認できない限り、マクロを有効にしません。不審な `.xlsm` や `.xlam` は開かず、公式の配布元や団体内の既知の連絡先で確認します。
5. 非公式のダウンロードサイト、海賊版・改変版の配布元、広告やポップアップから誘導された配布元は避けます。公式サイトまたは正規のアプリストアから入手します。
6. あるサービスから漏れた ID・パスワードが、他サービスへの不正ログインにも使われるためです。サービスごとに異なるパスワードを用い、多要素認証も有効にします。
7. 通知が本物かを公式サイト・公式アプリで確認し、ログイン履歴の端末・場所・時刻・操作に心当たりがあるかを確認めます。心当たりがなければ、パスワード変更、ログアウト・セッションの無効化、多要素認証の確認を行います。
8. URL を知る第三者が編集でき、内容の改ざん、削除、別資料の公開につながるおそれがあるためです。閲覧・編集の権限は必要の人に限定します。
9. 不要なカメラ、マイク、連絡先、位置情報、写真、入力内容などへのアクセスを許すと、必要以上の情報が収集・送信されるおそれがあるためです。機能に必要な権限かを確認します。
10. 警告内のボタンやリンクを押すこと、指定されたアプリを入れること、表示された電話番号へ連絡すること、認証情報やカード情報を入力することです。
11. メッセージ内のリンクや電話番号は使わず、公式サイト・公式アプリを自分で開くか、既知の連絡先へ別経路で確認します。急がせる文面ほど、まず立ち止まります。

第3章 個人情報

1. はい、氏名とメールアドレスの組合せは個人を識別できるため、個人情報に当たります。
2. 申込受付のために集めた連絡先を、説明していない別イベントの宣伝や第三者への提供に使うことなどです。
3. 漏えい、誤送信、権限設定ミス、不正アクセス、紛失などの対象になり、保有するほど管理責任と事故時の影響が増えるためです。

第4章 著作権

1. いいえ。ネットで公開されていることと、自由に利用できることは別です。許諾、ライセンス、または引用の要件を確認します。
2. 商用利用・改変・再配布の可否、クレジット表記、利用範囲、ライセンス条件、素材サイト自体の規約を確認します。
3. 他人の著作物、個人情報、内部情報、未公開情報、アカウント名、通知、URL、位置情報などが写り込むことです。
4. 最初の同意やライセンスが想定した利用目的・媒体・期間を超えることがあるためです。公開範囲や二次利用の条件を改めて確認します。
5. 一般に適切とはいえません。批評のために必要な範囲を超えやすく、引用部分が主従関係で「従」にならないおそれがあります。批評に必要な箇所にとり、出所を明示して引用の要件を満たす必要があります。

第5章 メール

1. BCC です。受信者同士にメールアドレスを見せないためです。
2. 実際の送信元メールアドレスは `info@example.com` です。表示上の `info@example.jp` と異なるため、正規のメールである可能性より、正規組織を装う不審なメールである可能性を強く疑います。正規の送信者が表示名と異なるドメインを使う合理的な利点は通常乏しいためです。メール内のリンクや返信先は使わず、公式サイト・既知の連絡先から確認します。
3. メール内のリンクや連絡先を使わず、公式サイト・公式アプリを自分で開くか、既知の連絡先へ別経路で確認することです。
4. まず送信停止、公開停止、共有リンクの無効化などで被害拡大を止め、速やかに関係者へ共有します。原因説明より先に止血を行います。

第6章 SNS・公開情報・テストデータ管理

1. スクリーンショット、転載、検索結果、キャッシュ、アーカイブ、通知、第三者による保存などが残るためです。
2. 個人名、メールアドレス、DM、通知、管理画面、未公開情報、URL、位置情報、ファイル名、ブラウザのタブなどの写り込みに注意します。
3. 開発・検証・デモ環境から個人情報や内部情報が漏れるおそれがあるためです。架空・匿名化・最小化したデータを使います。

第7章 生成 AI と情報管理

1. 個人情報、認証情報、未公開の内部資料などです。たとえば氏名・連絡先、パスワード・API キー、審査資料・議事録の草案は入力しません。
2. 生成 AI はもっともらしい誤情報や存在しない出典を出すことがあるためです。公開・送信・提出する前に、人が一次情報で確認します。
3. 一般に、出力を利用・公開した使用者または組織が責任を負います。「AI が作った」ことは確認や説明の責任を免れる理由にはなりません。
4. `.env`、秘密鍵、トークン、本番ログ、個人情報、DB ダンプ等を AI の読み取り対象から除外します。ツールが対応していれば AI 向け ignore や権限設定を使い、必要最小限のファイルだけを読ませます。採用する差分や実行するコマンドは人が確認します。

第8章 ファイル管理・バックアップ・ログ・Git

1. たとえば `20260718_事業名_文書種別_v01.docx` のように、日付・案件名・文書種別・版を一定の順序で付けます。団体内で規則を統一し、最終版のような曖昧な名前を避けます。
2. 削除コミットをしても、過去のコミット、クローン、フォーク等に `Secrets` が残り、取得・悪用され得るためです。まず無効化・再発行を行い、必要に応じて履歴の除去を検討します。

第9章 サーバ・コーディング関係のセキュリティ

1. ブラウザ側の表示や制限は利用者が変更でき、開発者ツールや直接リクエストで回避できるためです。重要な判断はサーバ側で行います。
2. 認証は「誰であるか」を確認すること、認可は「その人がその操作をしてよいか」を判断することです。
3. `.env` は平文ファイルであり、Git 混入、バックアップ流出、共有時の誤送信、配置先の権限ミスなどで読まれる可能性があるためです。
4. RLS はデータベースの行ごとに読み書き権限を制御する仕組みです。ただし、アプリ側の認可、接続権限、管理者権限、テスト、ログ監査の代わりにはなりません。
5. ユーザー入力をそのまま HTML として解釈せず、表示時にエスケープすることです。安全なテンプレート機構や CSP も組み合わせます。
6. CSRF トークンを使う、GET で状態変更しない、Cookie の `SameSite` 等を適切に設定する、重要操作には確認や再認証を入れることです。
7. SQL を文字列結合で組み立てず、プレースホルダーやプリペアドステートメントを使うことです。DB 権限の最小化も重要です。
8. 当該キー・トークンを無効化し、再発行することです。その後に影響範囲確認と再発防止を行います。
9. 本番の個人情報・内部情報・`Secrets` が開発環境や検証環境へ広がり、権限の弱い場所、ログ、外部委託先、デモ画面などから漏れる可能性があるためです。必要場合は匿名化・マスキングします。
10. 汚染されたパッケージが、インストール時や CI 実行時に `Secrets` やトークンを外部送信している可能性があるためです。パッケージを戻すだけでなく、影響範囲を確認し、漏えいした可能性のある認証情報を無効化・再発行します。

関西文芸交流会 情報リテラシ研修

2026 年 7 月 20 日 初版 (v1.0) 発行

開 発 関 西 文 芸 交 流 会 技 術 開 発 部
編 集 関 西 文 芸 交 流 会 技 術 開 発 部
発 行 関 西 文 芸 交 流 会

非売品・団体外秘

無断転載・転売を禁じます